

Investigating Volumetric Analysis of Constituents Of Brain Soft Tissue



by

**Daniyal Ahmed Gul (BCS223030)
Muhammad Hasan Nauman (BCS223033)
Hadia khan (BCS223216)**

Supervisor : Dr. Masroor Ahmed

**Proposal submitted as a
Final Year Project for
CS4912 Design Project**

**Department of Computer Sciences
Capital University of Science & Technology, Islamabad
February, 2026**

Table of Contents

Abstract	7
CHAPTER 1: Introduction	10
1.1 Project Introduction	10
1.2 Existing Solutions and Research Gap	12
1.3 Research Scope and Objectives	12
1.4 Tools and Technologies	13
1.5 Project Work Breakdown	15
1.6 Methodology	16
1.6.1 Overview of Methodology	16
1.6.2 Image Preprocessing	18
1.6.3 Feature Extraction and Reduction	18
1.6.4 Model Training and Classification	19
1.6.5 Clustering and Volumetric Estimation	20
1.6.6 Evaluation and Visualization	21
CHAPTER 2: Requirement Specification and Analysis	22
2.1 Functional Requirements	22
2.2 Non-Functional Requirements	23
2.3 Selected Functional Requirements	24
2.4 System Use Case Modeling	27
2.5 System Sequence Diagram	41
2.6 Domain Model	47
CHAPTER 3: System Design	48
3.1 Layer Definition	48
3.1.1 Presentation Layer	49
3.1.2 Application / Business Logic Layer	49
3.1.3 Data Layer	50
3.2 Software Architecture	50
3.3 Class Diagram	51
3.4 Sequence Diagram	54
3.4.1 Researcher Registration	54
3.4.2 Researcher Login	55
3.4.3 Upload MRI Image & Start Preprocessing	55
3.4.4 Feature Extraction, PCA, Classification & Volumetric Analysis	56
3.4.5 View & Download Volumetric Report	57
3.5 Entity Relationship Diagram	57
3.6 Database Schema	59

3.7 User Interface Design	60
CHAPTER 4: Implementation Issues	62
4.1 Coding Standards	62
4.1.1 Indentation	62
4.1.2 Declaration	62
4.1.3 Statement Standards	62
4.1.4 Naming Conventions	62
4.2 Development Environment	63
4.3 Database Management System	63
4.4 Software Description	64
4.4.1 User Authentication Module	65
4.4.2 Dashboard Interface	66
4.4.3 MRI Image Upload Module	67
4.4.4 Image Preprocessing Module	68
4.4.5 Output Visualization Module	69
CHAPTER 5: Software Testing	70
5.1 Testing Methodology	70
5.2 Testing Environment	70
5.3 Test Case: User Sign In	71
5.4 Test Case: Invalid Sign In	71
5.5 Test Case: MRI Image Upload	72
5.6 Test Case: Invalid File Upload	72
5.7 Test Case: Image Preprocessing	73
5.8 Test Case: Output Visualization	73

List of Tables

Table 1.1	Tools and Technologies.....	11
Table 1.2	Project Work Breakdown.	12
Table 2.1	Functional Requirements.....	19
Table 2.2	Non-Functional Requirements.....	20
Table 2.3	Selected Functional Requirements.....	21
Table 2.4	Register & Login.....	24
Table 2.5	Recover Password.....	25
Table 2.6	Upload MRI Images.....	26
Table 2.7	Validate Format.....	28
Table 2.8	Image Processing Pipeline.....	29
Table 2.9	Extract Features.....	30
Table 2.10	Train Model.....	31
Table 2.11	Perform Clustering.....	32
Table 2.12	Compute Volumetric Ratio.....	34
Table 2.13	Evaluate Model Performance.....	35
Table 2.14	Output Results.....	36
Table 3.1	Layers Definition.....	44
Table 5.1	User Sign In.....	70
Table 5.2	Invalid Login Attempt.....	71
Table 5.3	Upload MRI Image.....	72
Table 5.4	Unsupported File Format.....	72
Table 5.5	Preprocessing Execution.....	73
Table 5.6	Display Preprocessed Image.....	73

List of Figures

Figure 1	Overview of Methodology.....	16
Figure 2.1	Use Case Diagram.....	23
Figure 2.2	SSD (Register)	37
Figure 2.3	SSD (Login)	37
Figure 2.4	SSD (Recover Password)	38
Figure 2.5	SSD (Upload MRI Images)	38
Figure 2.6	SSD (Validate Format)	39
Figure 2.7	SSD (Image Processing Pipeline)	39
Figure 2.8	SSD (Extract Features)	39
Figure 2.9	SSD (Train Model)	40
Figure 2.10	SSD (Perform Clustering)	40
Figure 2.11	SSD (Compute Volumetric Ratio)	41
Figure 2.12	SSD (Evaluate Model Performance)	41
Figure 2.13	SSD (Output Results)	42
Figure 3.1	Software Architecture Diagram.....	46
Figure 3.2	UML Class Diagram.....	47
Figure 3.3	Researcher Registration.....	50
Figure 3.4	Researcher Login.....	51
Figure 3.5	Upload MRI Image and Start Processing.....	51
Figure 3.6	Feature Extraction, PCA, Classification & Volumetric Analysis.....	52
Figure 3.7	View & Download Report.....	53
Figure 3.8	ERD Brain Soft Tissue Volumetric Analysis System.....	54
Figure 3.9	Database Schema for Brain Soft Tissue Volumetric Analysis.....	55
Figure 4.1	Dashboard Interface.....	66
Figure 4.2	MRI Image Upload Interface.....	67
Figure 4.3	Image Preprocessing Output.....	68
Figure 4.4	Output Visualization Screen.....	69

Abstract

The human brain is a structurally intricate organ composed of heterogeneous soft tissues principally white matter, gray matter, and cerebrospinal fluid whose volumetric proportions serve as crucial indicators of both neurological health and disease progression. Precise volumetric analysis of these brain tissue constituents is essential for diagnosing disorders such as Alzheimer’s disease, multiple sclerosis, and brain tumors, as well as for understanding normal aging processes. Despite major advances in medical imaging, current volumetric quantification techniques are hindered by high computational cost, inconsistent accuracy across scanners, and dependence on manual segmentation. The proposed project, “Investigating Volumetric Analysis of Constituents of Brain Soft Tissue,” seeks to develop a data-driven, machine-learning enabled framework that automates volumetric analysis using two-dimensional (2D) magnetic resonance imaging (MRI) slices reconstructed into volumetric estimates. By fusing image-processing algorithms, feature-extraction techniques, and predictive learning models, this research aims to create a reliable, efficient, and clinically interpretable pipeline that supports quantitative brain-tissue profiling.

At its foundation, this study addresses a critical gap between traditional volumetric segmentation and intelligent image analytics. Conventional methods such as manual delineation or atlas-based segmentation are accurate but impractical for large datasets and clinical throughput. The proposed system leverages public imaging repositories such as Kaggle Brain MRI Dataset, The Cancer Imaging Archive (TCIA), and Open Access Series of Imaging Studies (OASIS) to construct a robust and diverse database. These repositories collectively offer MRI slices from patients with varied pathological and demographic backgrounds, ensuring generalizable outcomes. Data will be imported into MATLAB, where preprocessing operations including image resizing, denoising, skull stripping, histogram equalization, and intensity normalization will standardize pixel intensities and suppress imaging artifacts. The processed images will then undergo region-of-interest extraction to isolate brain tissues while removing non-essential background structures. Each image will be mapped to metadata containing patient demographics and clinical conditions, creating a comprehensive input matrix for downstream analysis.

Following preprocessing, the system enters its computational core an adaptive modeling and learning stage. Feature vectors will be generated through texture-based descriptors (GLCM, LBP, Haralick features) and shape-based parameters (area, eccentricity, compactness). These handcrafted radiomic features will be augmented by deep-feature embeddings derived from Convolutional Neural Networks (CNNs) such as EfficientNet-B0 or MobileNetV2, pretrained on large-scale medical imaging datasets and fine-tuned for MRI brain segmentation. Comparative experiments will also evaluate Decision Tree and Random Forest classifiers using extracted features, enabling an interpretability benchmark against deep models. A hybrid feature matrix will then be constructed, integrating statistical, radiomic, and learned representations. Dimensionality reduction techniques such as Principal Component Analysis (PCA) and t-SNE visualization will

minimize redundancy, enhance computational efficiency, and reveal intrinsic clustering structures within the tissue-type distributions.

The novelty of this research lies in its integration of classical machine learning and deep learning for quantitative volumetric computation from 2D imaging data. Instead of relying on full 3D scans or manually drawn masks, the framework reconstructs volumetric estimates from sequential slices using image-based intensity mapping and clustering of segmented tissue regions. Through unsupervised clustering (K-means, DBSCAN), the algorithm differentiates anatomical constituents based on pixel intensity histograms and spatial continuity, while the machine learning classifier assigns tissue labels to each cluster. The aggregated slice-level volumes are computed into 3D estimates using voxel scaling, generating accurate relative percentages of white matter, gray matter, and cerebrospinal fluid volumes.

The project also integrates a comparative evaluation module to determine which algorithmic combination yields optimal accuracy, efficiency, and interpretability. Decision Trees will be evaluated for feature importance transparency, CNNs for representation depth, and hybrid models for trade-off balance. Model performance will be validated on hold-out datasets and cross-institutional test sets, ensuring generalization across varying acquisition protocols. Evaluation metrics will include accuracy, sensitivity, specificity, F1 score, Dice similarity coefficient, and Intersection over Union (IoU). These indicators will provide statistical grounding for model selection and demonstrate reproducible clinical validity.

To ensure interpretability and ethical deployment, the proposal emphasizes explainable AI (XAI) techniques such as Grad-CAM visualizations, feature importance plots, and saliency maps, allowing clinicians to verify model outputs visually. The research further incorporates a data governance framework that respects patient privacy and ensures all data sources comply with ethical standards. The system will use de-identified, publicly available datasets and maintain secure local storage under institutional approval.

In practical application, the proposed model offers extensive utility in both clinical and research contexts. Clinically, the algorithm could assist radiologists by generating preliminary tissue volume ratios that support diagnoses of brain atrophy, edema, or tumor infiltration. For research institutions, the platform can accelerate large-scale neuroimaging studies, reducing human bias in segmentation tasks and enabling statistically consistent data analytics across cohorts. In the long term, this work aims to serve as a foundation for automated diagnostic systems that quantify brain tissue dynamics over time, helping clinicians monitor treatment outcomes and disease progression.

The project's deliverables include:

1. **A standardized brain MRI dataset suitable for volumetric research.**
2. **A preprocessing and segmentation toolkit implemented in MATLAB and Python.**
3. **A comparative analysis report of Decision Tree vs. CNN vs. hybrid models.**
4. **An interactive dashboard for visualizing volumetric proportions and performance metrics.**
5. **A published dataset and trained model repository for reproducible research.**

The project resources will focus on dataset curation, computational modeling, and validation experiments to ensure scientific rigor and reproducibility. The expected outcomes include a validated volumetric analysis framework that achieves over 90% Dice accuracy in tissue classification and significantly reduces manual segmentation effort by over 70%. This directly contributes to improving diagnostic speed, reproducibility, and data accessibility in neuroimaging research.

In conclusion, “*Investigating Volumetric Analysis of Constituents of Brain Soft Tissue*” aims to pioneer a balanced fusion of conventional radiomics and modern deep learning to achieve scalable, interpretable volumetric quantification from 2D brain images. Through an optimized pipeline encompassing dataset acquisition, image preprocessing, segmentation, feature engineering, dimensionality reduction, clustering, and machine learning classification, the project promises a novel benchmark for volumetric accuracy and automation in neuroimaging. Its success will not only validate the potential of hybrid computational approaches in medical imaging but also lay the groundwork for AI-driven clinical tools that enhance precision, efficiency, and accessibility in the evaluation of human brain structure.

CHAPTER-1

Introduction

1.1 Project Introduction

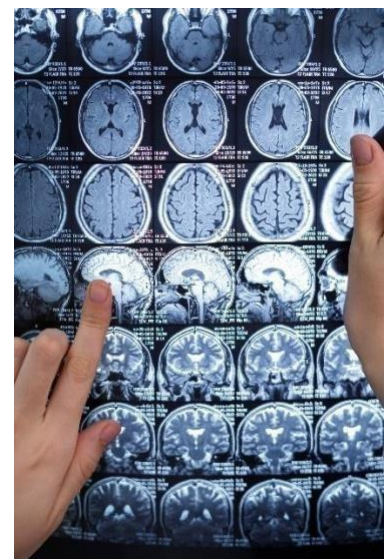
The human brain is a structurally complex organ composed of distinct soft tissue constituents, including gray matter, white matter, and cerebrospinal fluid (CSF). These components play vital roles in neural communication, cognitive processing, and physiological regulation. Variations in their relative volumes are closely associated with neurological disorders such as Alzheimer's disease, Parkinson's disease, brain tumors, and multiple sclerosis. Therefore, the quantitative volumetric analysis of these tissues provides valuable diagnostic and research insights into disease detection, progression, and treatment response.

Traditional volumetric analysis methods rely heavily on manual segmentation or semi-automated tools that require expert intervention. These methods, while accurate, are time-consuming, prone to inter-operator variability, and computationally expensive when dealing with large MRI datasets. With the evolution of artificial intelligence, especially machine learning (ML) and deep learning (DL), there is now a significant opportunity to automate this analysis process with higher precision and efficiency.

interpretability, speed, and accuracy bridging the gap between traditional radiomics and modern AI-driven medical imaging.

1.2 Existing Solutions and Research Gap

Several tools currently exist for brain tissue segmentation and volumetric computation, including FreeSurfer, FSL, SPM, and 3D U-Net-based architectures. While these frameworks have advanced the field of



neuroimaging, they often require 3D volumetric data, high-end computational resources, and manual correction by experts. Moreover, their performance can degrade across different MRI scanners and patient demographics due to inconsistencies in acquisition protocols and noise levels.

Recent research in deep-learning-based segmentation (e.g., U-Net, ResNet, and VGG models) has shown impressive accuracy but often sacrifices model interpretability and efficiency. Classical methods such as K-means clustering, Expectation-Maximization, and Decision Trees offer simpler, explainable results but lack the adaptive learning ability required for heterogeneous MRI data. Thus, there remains a need for a hybrid approach that combines the interpretability of classical ML with the accuracy and learning depth of CNNs.

The proposed research addresses this gap by employing 2D MRI slices from publicly available datasets (Kaggle, TCIA, OASIS) to reconstruct volumetric estimates without requiring full 3D scans. This approach significantly reduces computational complexity while maintaining volumetric accuracy through advanced preprocessing, feature extraction, and hybrid modeling.

1.3 Research Scope and Objectives

This project focuses on building a robust, data-driven pipeline for volumetric analysis of brain soft tissues. The scope includes data collection, image preprocessing, feature extraction, dimensionality reduction, model training, and volumetric computation.

Research Objectives:

- 1. To construct a diverse MRI dataset from publicly available repositories (Kaggle, TCIA, OASIS).**
- 2. To design a MATLAB-based preprocessing pipeline that performs denoising, normalization, skull stripping, and contrast enhancement.**

3. **To extract and integrate texture-based, shape-based, and deep CNN features into a hybrid feature matrix.**
4. **To apply dimensionality reduction (PCA, t-SNE) for optimized model performance.**
5. **To compare Decision Tree, Random Forest, and CNN models for tissue segmentation accuracy.**
6. **To compute volumetric ratios of gray matter, white matter, and CSF from reconstructed 2D slices.**
7. **To evaluate performance using Dice Similarity Coefficient (DSC), Intersection over Union (IoU), and F1-score metrics.**

This structured workflow aims to deliver a computationally efficient and clinically interpretable volumetric analysis system that can be extended to disease-specific segmentation in future research.

1.4 Tools and Technologies

The implementation will leverage both MATLAB and Python environments to ensure compatibility with advanced image processing and ML libraries. MATLAB will handle preprocessing, while Python will be used for training, testing, and evaluating machine-learning models.

Table 1: Tools and Technologies

Category	Technology/Tool	Purpose/Function
Programming Languages	MATLAB, Python	Image preprocessing, feature extraction, and ML model development
Libraries/Frameworks	TensorFlow, Scikit-learn, OpenCV	Model training, feature engineering, and clustering
Datasets	Kaggle Brain MRI, TCIA, OASIS	Source of 2D MRI brain images
Algorithms	Decision Tree, CNN, PCA, K-Means	Classification, segmentation, feature reduction
Evaluation Metrics	Accuracy, Dice Score, IoU, F1-score	Performance validation
Hardware	GPU-Enabled Machine	Accelerated deep-learning model training

1.5 Project Work Breakdown

Table 2: Project Work Breakdown

Phase	Duration	Focus Area	Expected Outcomes
Phase 1	Months 1–2	Literature Review & Dataset Collection	Detailed study of segmentation methods and dataset preparation
Phase 2	Months 3–4	Image Preprocessing & ROI Extraction	MATLAB-based preprocessing and skull stripping
Phase 3	Months 5–6	Feature Extraction & Model Training	Generation of feature matrix, training of Decision Tree and CNN models
Phase 4	Months 7–8	Feature Reduction & Volumetric Computation	PCA/t-SNE for optimization and tissue volume estimation
Phase 5	Months 9–10	Model Evaluation & Comparative Analysis	Model validation and hybrid accuracy benchmarking
Phase 6	Months 11–12	Final Report & Documentation	Compilation of results, documentation, and presentation

1.6 Methodology

1.6.1 Overview of Methodology

The proposed research follows a systematic workflow designed to automate the volumetric analysis of brain soft tissue constituents using MRI data. The methodology combines classical image-processing techniques with modern machine-learning and deep-learning approaches to achieve accurate segmentation, clustering, and volumetric computation.

The process begins with the acquisition of publicly available 2D MRI datasets such as Kaggle, TCIA, and OASIS. These datasets provide diverse imaging conditions and subject demographics, ensuring model generalization. The MRI slices are preprocessed in MATLAB to remove noise, normalize intensities, and isolate the brain region. After preprocessing, relevant features are extracted—both hand-crafted (texture and shape-based) and deep features from CNN embeddings. The extracted features are then reduced using Principal Component Analysis (PCA) or t-Distributed Stochastic Neighbor Embedding (t-SNE) to optimize computational efficiency and minimize redundancy. Two models are evaluated for tissue classification: a Decision Tree (for interpretability) and a Convolutional Neural Network (CNN) (for accuracy). The classified outputs are clustered to estimate the volumetric distribution of gray matter, white matter, and cerebrospinal fluid (CSF). Finally, results are validated using statistical metrics such as Dice Similarity Coefficient (DSC), Intersection over Union (IoU), accuracy, and F1-score, followed by

visualization of quantitative results.

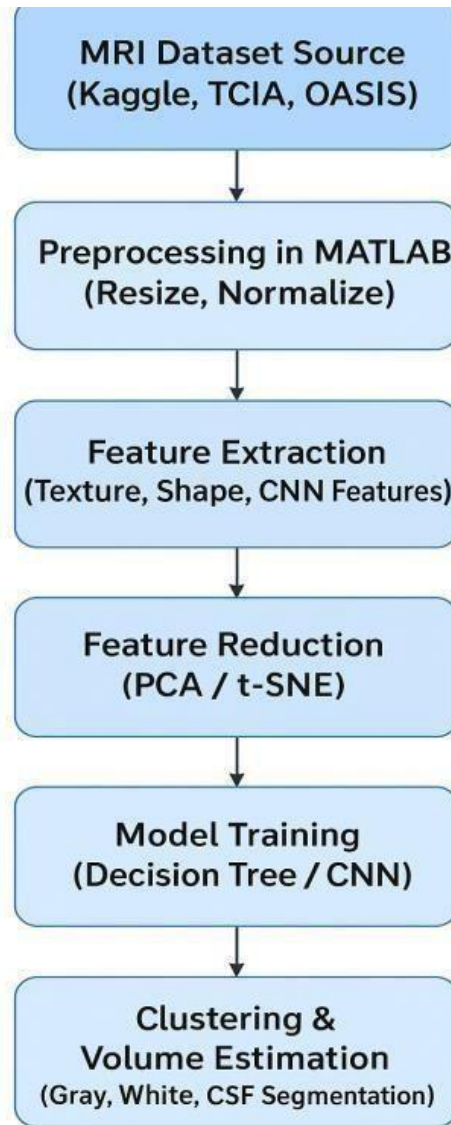


Figure 1: Overview of Methodology

1.6.2 Image Preprocessing

Image preprocessing is a fundamental stage to ensure MRI slices are clean, standardized, and ready for analysis. Variability in scanner settings, patient motion, and noise can distort pixel intensity and affect segmentation accuracy; hence this step enhances data quality.

The process begins by importing the MRI data using MATLAB functions (`dicomread`, `niftiread`) and converting all images to grayscale. Uniformity in image size is maintained through resizing and cropping, ensuring consistent input dimensions for the models. Noise reduction is applied via Gaussian or median filters to smooth image regions while preserving edges. Next, skull stripping removes non-brain tissue using morphological operations and intensity-based thresholding, retaining only the brain



region of interest (ROI). Histogram equalization enhances contrast, making gray and white matter boundaries clearer, while intensity normalization adjusts pixel ranges for uniform brightness and contrast.

Finally, binary segmentation masks are generated for each slice, isolating meaningful brain regions and preparing data for feature extraction.

1.6.3 Feature Extraction and Reduction

Once the MRI slices are preprocessed, essential image characteristics are extracted to form a

feature matrix that numerically represents tissue structures.

1. Texture features:

These describe intensity variations within brain tissues using methods like Gray-Level Co-Occurrence Matrix (GLCM), Local Binary Patterns (LBP), and Haralick features, which capture contrast, correlation, and homogeneity.

2. Shape features:

Geometric attributes such as area, eccentricity, and compactness provide information about tissue morphology and spatial distribution.

3. Deep CNN features:

Pretrained networks (e.g., MobileNetV2, EfficientNet-B0) are used to extract high-level representations from MRI slices, enabling the system to learn complex patterns automatically.

The resulting hybrid feature set may contain hundreds of variables. Therefore, dimensionality reduction techniques such as PCA and t-SNE are applied to remove redundant or correlated features, preserving only the most informative ones. This step enhances both speed and model interpretability.

1.6.4 Model Training and Classification

The refined feature matrix serves as the input for classification models. Two learning paradigms are compared:

- **Decision tree model:**

A rule-based algorithm that splits features according to thresholds derived from information gain or Gini index. It offers transparent decision rules, making the output

interpretable for clinicians.

- **Convolutional neural network (CNN):**

A multi-layer architecture capable of automatically learning hierarchical spatial features from MRI images. The CNN consists of convolutional layers (for feature detection), pooling layers (for dimensionality reduction), and fully connected layers (for classification). Transfer learning is applied by fine-tuning pretrained networks to adapt to MRI data.

Both models are trained and validated using cross-validation strategies. The Decision Tree serves as a baseline, while the CNN is expected to yield higher segmentation accuracy.

1.6.5 Clustering and Volumetric Estimation

After classification, the segmented tissue regions are clustered based on intensity and spatial proximity. Unsupervised algorithms such as K-Means or DBSCAN group pixels belonging to gray matter, white matter, and CSF. Each cluster's pixel count is multiplied by voxel dimensions to compute volumetric ratios:

$$V_i = \frac{N_i}{N_{total}} \times 100$$

where V_i is the percentage volume of tissue i , N_i is the number of pixels in that cluster, and N_{total} is the total pixel count.

The final volumetric report expresses the relative percentage of each tissue type, enabling quantitative comparisons across subjects.

1.6.6 Evaluation and Visualization

Model performance is assessed using quantitative metrics:

- **Accuracy, Precision, Recall, F1-Score measure classification correctness.**
- **Dice Similarity Coefficient (DSC) and Intersection over Union (IoU) evaluate overlap between predicted and ground-truth regions.**

Results are visualized through bar charts, confusion matrices, and 2D color-coded tissue maps to depict the volumetric proportions of gray, white, and CSF regions.

These visualizations help verify whether the computed volumetric estimates align with known anatomical expectations.

2 Chapter 2: Requirement Specification and Analysis

Requirements analysis is a process of determining user expectations for a new or modified product. These features, called requirements, must be quantifiable, relevant and detailed. In software engineering, such requirements are often called functional specifications. In Chapter 2 we will enlist the functional and non-functional requirements and model functional requirements in the form of use case model.

2.1 Functional Requirements

A functional requirement defines a function of a system or its component. Functional requirements may include technical details, data manipulation, processing, and other specific functionalities that define what a system is supposed to accomplish.

Table 3: Functional Requirements

S. No	Functional Requirement	Type	Status
1	Researcher registers and logs into the system	Core	Pending
2	Researcher uploads MRI images to the system	Core	Pending
3	System performs automated preprocessing (denoising, skull stripping, normalization, histogram equalization)	Core	Pending
4	System extracts texture features (GLCM, LBP) and shape features (area, eccentricity) from preprocessed images	Core	Pending

5	System generates deep CNN features using pretrained models (MobileNetV2 or EfficientNet-B0)	Core	Pending
6	System applies PCA for dimensionality reduction	Intermediate	Pending
7	Researcher selects and trains classification model (Decision Tree or CNN)	Core	Pending
8	System performs K-Means clustering to differentiate tissue types	Core	Pending
9	System computes volumetric ratios of gray matter, white matter, and CSF using voxel scaling	Core	Pending
10	System evaluates model performance using Dice Score, IoU, Accuracy, and F1-Score	Intermediate	Pending
11	System generates volumetric analysis report with tissue percentages and visualizations	Core	Pending
12	Researcher views and exports segmentation results and trained models	Intermediate	Pending
13	System provides password recovery functionality	Intermediate	Pending
14	User logs out from the system	Core	Pending

2.2 Non-Functional Requirements

A non-functional requirement specifies the criteria used to judge the operation of a system rather than specific behaviors. They are contrasted with functional requirements that define specific actions or processes.

Table 4: Non-Functional Requirements

S. No.	Non-Functional Requirements	Category
1	The user interface should be intuitive and easy to use for medical researchers	Usability
2	The system should provide feedback during image upload and processing	Performance
3	Preprocessing should complete within 5 minutes for standard MRI datasets	Performance
4	The system must support secure login with encrypted passwords	Security
5	All uploaded MRI data must be de-identified and stored securely	Security
6	The system should process and preprocess images with high accuracy	Reliability
7	Model training should complete within 2-4 hours on standard hardware	Performance
8	Trained models should achieve at least 85% Dice Similarity Coefficient	Reliability
9	The system should handle at least 10-20 concurrent users	Scalability
10	Database operations should execute within 3 seconds	Performance
11	The system should maintain detailed logs for reproducibility	Maintainability
12	The system architecture should be modular for easy updates	Maintainability
13	Data should be backed up regularly to prevent loss	Security
14	The system should work on Windows and Linux platforms	Portability

2.3 Selected Functional Requirements

The following list contains the functional requirements selected for the current development iteration of the brain volumetric analysis system.

Table 5: Selected Functional Requirements

S. No	Functional Requirements	Type
1	Researcher registers and logs into the system	New
2	Researcher uploads MRI images to the system	New
3	System performs automated preprocessing pipeline	New

2.4 System Use Case Modeling

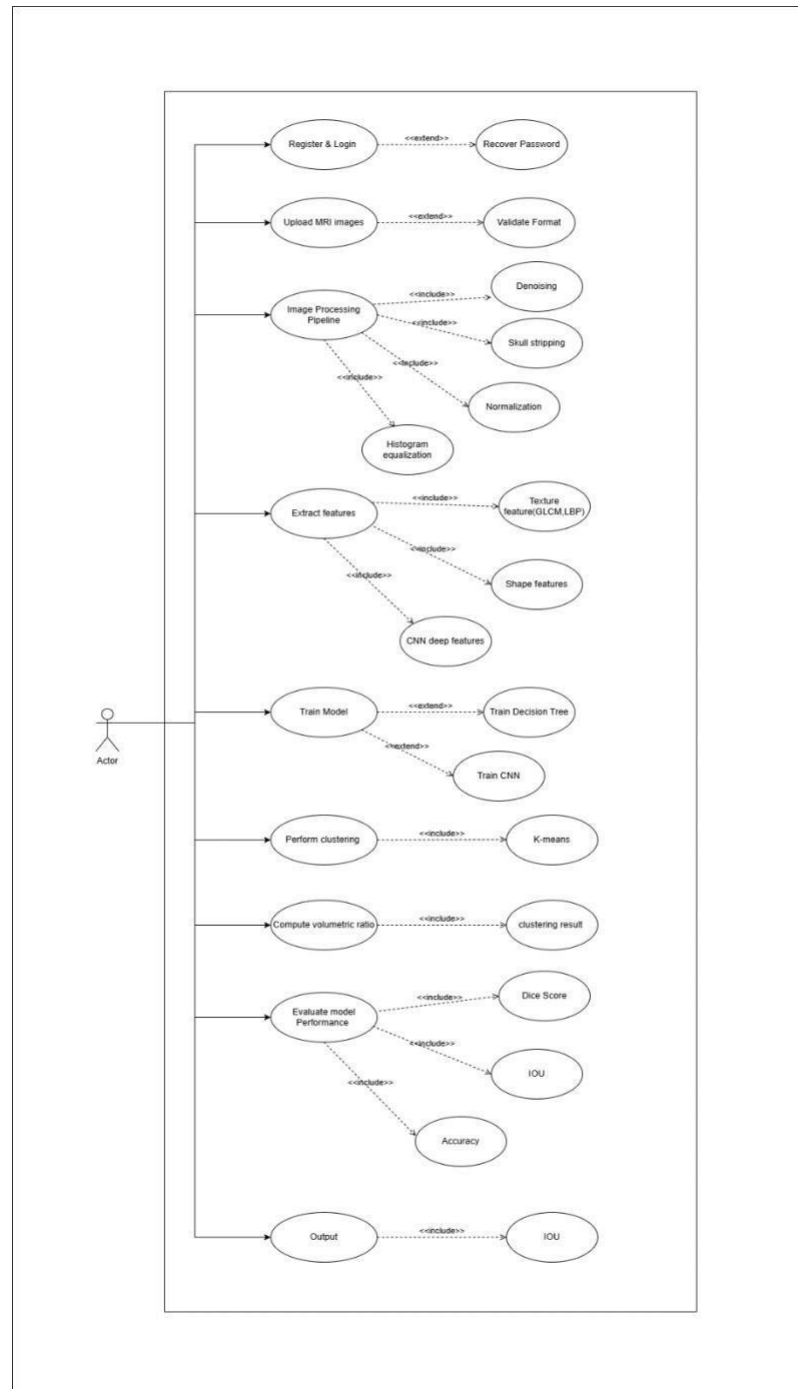


Figure 2: Use Case Diagram

Table 6: Register & Login

Use Case ID:	UC 1		
Use Case Name:	Register & Login		
Created By:	Hadia Khan	Last Updated By:	Daniyal Ahmed
Date Created:	11/19/2025	Last Revision Date	11/9/2025
Actors:	User (Researcher / System Operator)		
Description:	This use case allows a user to register a new account and log into the system. Logging in enables access to dataset uploading, processing, model training, and evaluation modules.		
Trigger:	User wants to access the system.		
Precondition	• System is running. • User has a valid account (for login). • Internet or local access to system UI.		
Postcondition:	• User is authenticated and can access the dashboard.		
Normal Flow:	Actor	System	
	Opens login/registration page		
	Enters credentials	Validates inputs Checks database for credentials	

		Grants access Redirects to dashboard
Alternative Flows:	• A1: <i>Invalid credentials</i> → <i>System prompts retry.</i> • A2: <i>New user selects Register</i> → <i>enters details</i> → <i>Account created.</i>	
Exceptions:	• E1: Database connection failure. • E2: Missing required input fields.	

Table 7: Recover Password

Use Case ID:	UC 2		
Use Case Name:	Recover Password		
Created By:	Daniyal Ahmed	Last Updated By:	Raja Hasan
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	User (Researcher / System Operator)		
Description:	Allows users to recover their account credentials in case they forget their password..		
Trigger:	User clicks “Forgot Password.”		

Precondition	User already has an account.	
Postcondition:	Password reset link sent OR temporary password generated.	
Normal Flow:	Actor	System
	1. Requests password recovery	2. Asks for email
	3. Provides email	4. Validates email and sends reset link
Alternative Flows:	A1: User enters wrong email → System shows “email not found.”	
Exceptions:	E1: Email server not reachable.	

Table 8: Upload MRI Images

Use Case ID:	UC 3		
Use Case Name:	Upload MRI Images		
Created By:	Hadia Khan	Last Updated By:	Raja hasan
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	User		
Description:	User uploads MRI slices for preprocessing, feature extraction, and volumetric analysis.		
Trigger:	User selects “Upload Dataset.”		

Precondition	• User logged in. • Dataset available on local system.	
Postcondition:	• Images stored and validated.	
Normal Flow:	Actor	System
	1. Selects MRI images	
		2. Checks file format 3. Stores dataset 4. Displays upload success
Alternative Flows:	• A1: Multiple images uploaded together. • A2: Partial upload failure.	
Exceptions:	• E1: Invalid file format. • E2: Corrupted files.	

Table 9: Validate Format

Use Case ID:	UC 4		
Use Case Name:	Validate Format		
Created By:	Hadia Khan	Last Updated By:	Raja Hasan
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	System		
Description:	Confirms whether uploaded images follow required format (NIfTI, DICOM, PNG, etc.)		
Trigger:	File upload event.		
Precondition	File selected.		
Postcondition:	File accepted OR rejected.		
Normal Flow:	Actor	System	
		System automatically reads metadata, checks headers, and	
		validates MRI slice dimensions.	
Alternative Flows:	A1: User chooses to convert file to supported format.		

Exceptions:	E1: Unsupported format detected.
--------------------	--

Table 10: Image Processing Pipeline

Use Case ID:	UC 5		
Use Case Name:	Image Processing Pipeline		
Created By:	Daniyal Ahmed	Last Updated By:	Raja Hasan
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	System (auto-process), User (initiates)		
Description:	Executes preprocessing steps: denoising, skull stripping, histogram equalization, and normalization.		
Trigger:	User clicks “Process Images.”		
Precondition	MRI images uploaded.		
Postcondition:	Cleaned images stored and passed to feature extraction.		
Normal Flow:	Actor	System	
	1. Starts preprocessing	2. Denoises image	

		3. Performs skull stripping 4. Applies histogram equalization 5. Normalizes intensities 6. Saves processed results
Alternative Flows:	A1: User selects only specific preprocessing steps.	
Exceptions:	- E1: Skull stripping fails on low-contrast image. - E2: Processing pipeline crash.	

Table 11: Extract Features

Use Case ID:	UC 6		
Use Case Name:	Extract Features		
Created By:	Hadia Khan	Last Updated By:	Daniyal Ahmed
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	User, System		
Description:	Extracts handcrafted (GLCM, LBP, shape) and deep features (CNN embeddings).		
Trigger:	User selects "Extract Features."		

Precondition	Preprocessed images available.	
Postcondition:	Feature matrix is generated.	
Normal Flow:	Actor	System
	1. Requests feature extraction	2. Computes texture features (GLCM, LBP)
		3. Computes shape features 4. Extracts CNN deep features 5. Stores hybrid feature matrix
Alternative Flows:	- A1: User selects “Only CNN features.” - A2: User selects “Only handcrafted features.”	
Exceptions:	- E1: CNN model weights missing. - E2: Low RAM causes extraction failure.	

Table 12: Train Model

Use Case ID:	UC 7
Use Case Name:	Train Model

Created By:	Raja Hasan	Last Updated By:	Daniyal Ahmed
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	User, System		
Description:	Train either a Decision Tree model or CNN using the extracted features.		
Trigger:	User selects “Train Model.”		
Precondition	Feature matrix exists.		
Postcondition:	Model is trained and saved.		
Normal Flow:	Actor	System	
	1. Selects model type	2. Loads dataset	
		3. Performs training 4. Validates model 5. Saves model	
Alternative Flows:	A1: User selects both models for comparison.		
Exceptions:	- E1: GPU unavailable for CNN training. - E2: Insufficient training samples.		

Table 13: Perform Clustering

Use Case ID:	UC 8		
Use Case Name:	Perform Clustering		
Created By:	Daniyal Ahmed	Last Updated By:	Hadia Khan
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	System		
Description:	Applies K-Means to group pixels into gray matter, white matter, and CSF.		
Trigger:	Model output needs grouping.		
Precondition	Model output or segmentation mask available.		
Postcondition:	Cluster labels stored.		
Normal Flow:	Actor	System	
	System runs clustering → assigns tissue labels → generates segmentation output		
Alternative Flows:	A1: User selects DBSCAN instead of K-Means.		

Exceptions:	• E1: Poor intensity contrast → inaccurate clustering.
--------------------	--

Table 14: Compute Volumetric Ratio

Use Case ID:	UC 9		
Use Case Name:	Compute Volumetric Ratio		
Created By:	Raja hasan	Last Updated By:	Daniyal Ahmed
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	System		
Description:	Computes volumetric ratio using cluster pixel counts and voxel scaling.		
Trigger:	Clustering finishes.		
Precondition	Clustering complete.		
Postcondition:	Percentage volumes for GM/WM/CSF generated.		
Normal Flow:	Actor	System	
	System counts pixels → applies volume formula → stores quantitative results.		
Alternative Flows:			

Exceptions:	E1: Missing slice metadata.
--------------------	---

Table 15: Evaluate Model Performance

Use Case ID:	UC 10		
Use Case Name:	Evaluate Model Performance		
Created By:	Hadia Khan	Last Updated By:	Raja Hasan
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	User, System		
Description:	Computes DSC, IoU, Accuracy, F1.		
Trigger:	User selects “Evaluate Model.”		
Precondition	Predictions + ground truth available.		
Postcondition:	Evaluation scores saved and visualized.		
Normal Flow:	Actor	System	
	System loads predicted masks → compares with ground truth → generates scores → shows graphs.		

Alternative Flows:	A1: User selects custom metric set.
Exceptions:	E1: Ground truth missing or mismatched.

Table 16: Output Results

Use Case ID:	UC 11		
Use Case Name:	Output Results		
Created By:	Raja Hasan	Last Updated By:	Daniyal Ahmed
Date Created:	11/9/2025	Last Revision Date	11/9/2025
Actors:	User, System		
Description:	Displays volumetric ratios, segmentation overlays, evaluation metrics.		
Trigger:	Evaluation finished.		
Precondition	Evaluation complete.		
Postcondition:	Final outputs shown in dashboard.		
Normal Flow:	Actor	System	
	System displays graphs, numeric values, and segmentation maps.		

Alternative Flows:	
Exceptions:	• E1: Visualization component fails.

2.5 System Sequence diagram

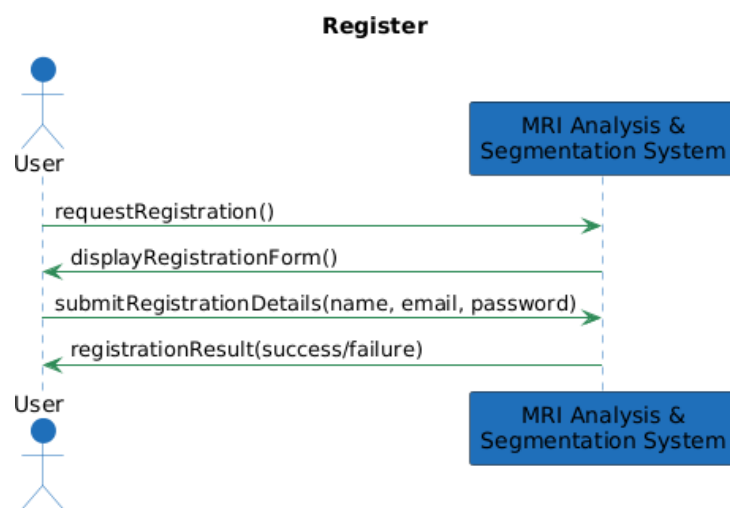


Figure 3: SSD(Register)

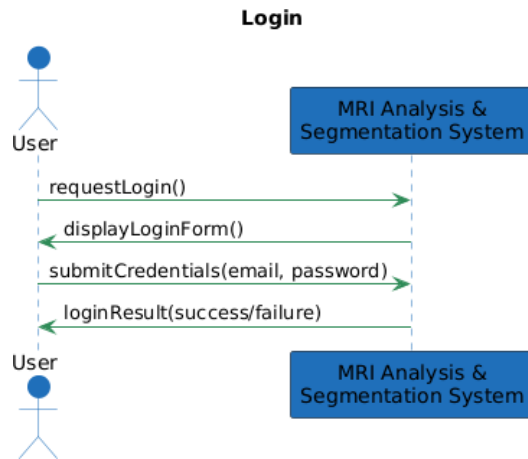


Figure 4: SSD(Login)

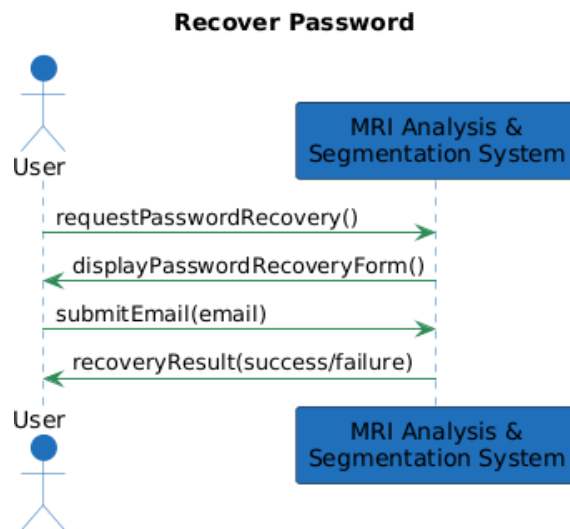


Figure 5: SSD (Recover Password)

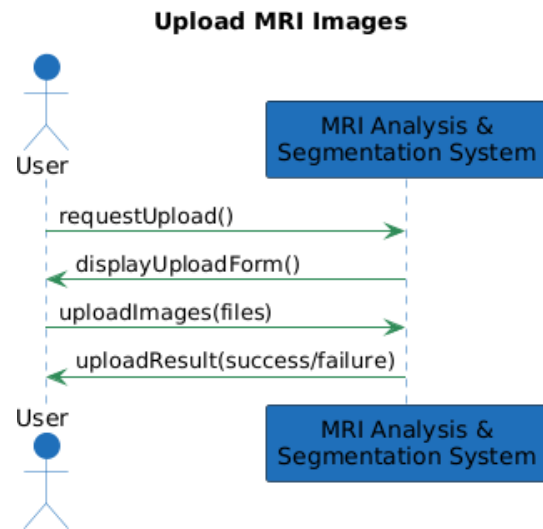


Figure 6: SSD (Upload MRI Images)

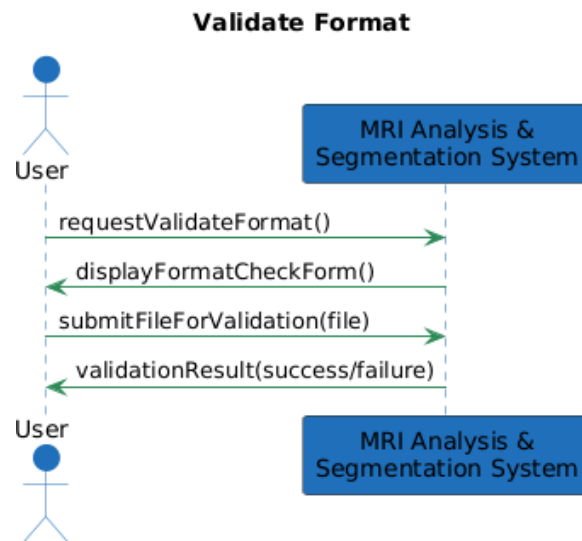


Figure 7: SSD (Validate Form)

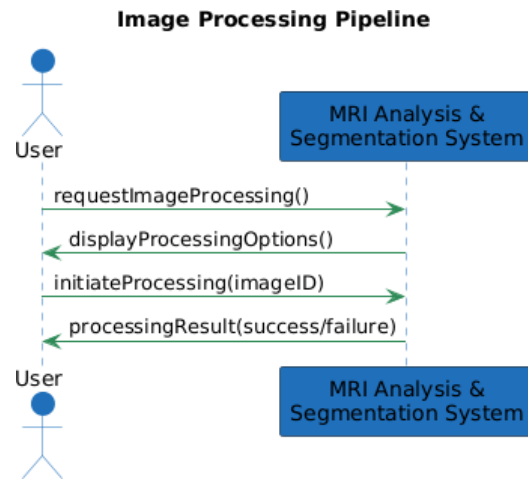


Figure 8: SSD (Image Processing Pipeline)

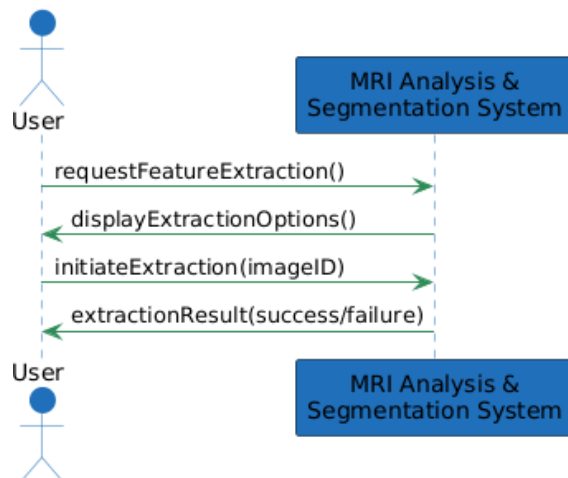


Figure 9: SSD (Extract Features)

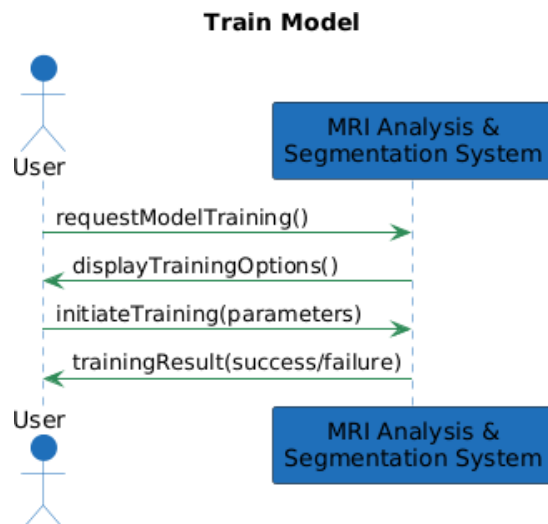


Figure 10: Train Model

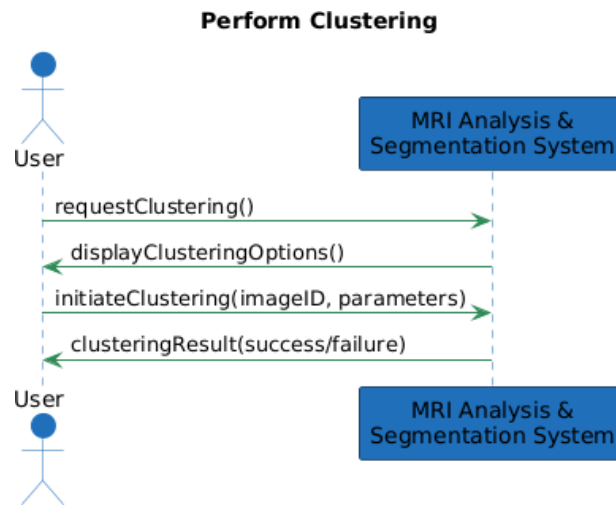


Figure 11: Perform Clustering

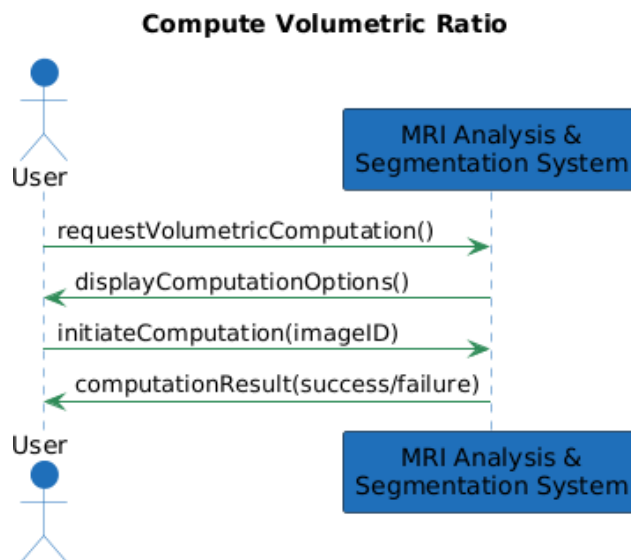


Figure 12: SSD (Compute Volumetric Ratio)

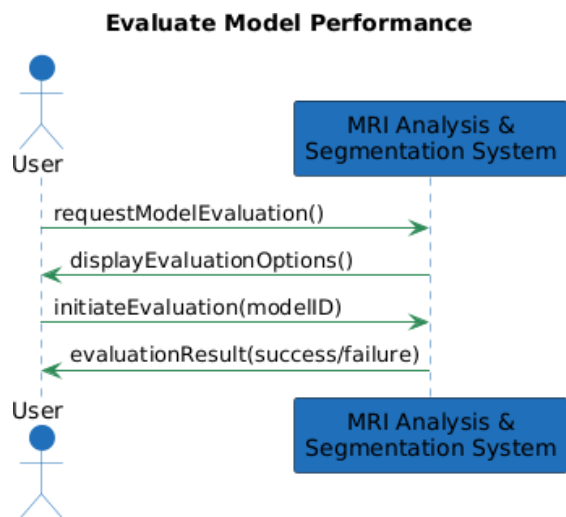


Figure 14: SSD (Evaluate Model Performance)

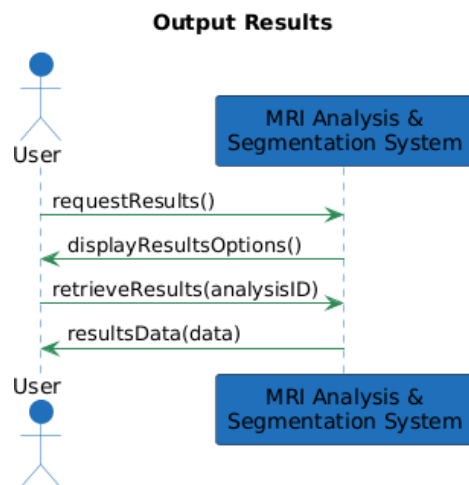


Figure 13: SSD (Output Results)

2.6 Domain Model

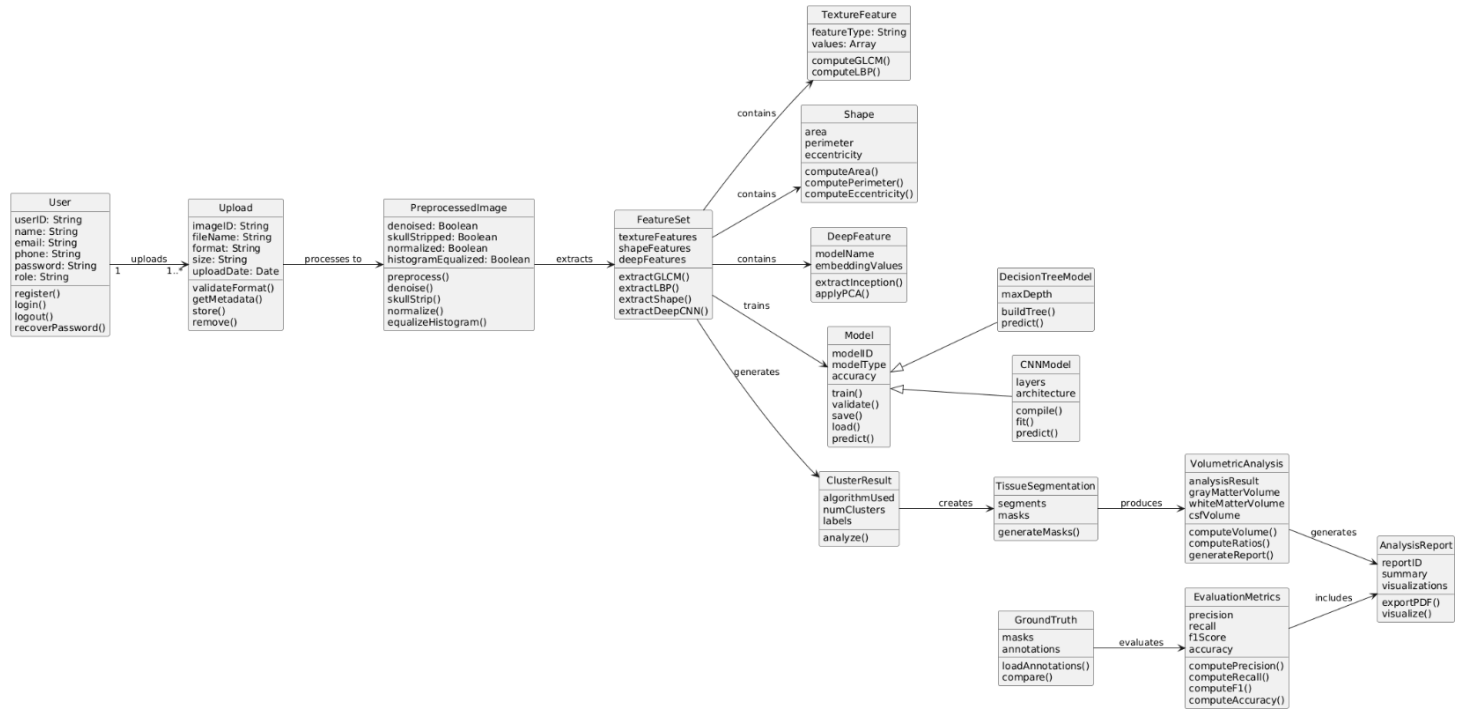


Figure 15: Domain Model

3 Chapter-3 :System Design

The proposed system follows a structured end-to-end pipeline that converts raw MRI slices into meaningful volumetric information about brain tissues. The process begins with image preprocessing, where the MRI data is cleaned through denoising, skull stripping, histogram equalization, and intensity normalization to ensure consistency across all slices. After preprocessing, the system performs segmentation and tissue isolation to remove irrelevant background and focus solely on the brain region. From the segmented images, multiple types of features are extracted, including texture descriptors, shape characteristics, and deep features obtained through pretrained CNN models. These features are further optimized using dimensionality reduction techniques such as PCA to remove noise and redundancy. The refined feature set is then passed to machine-learning and deep-learning classifiers—such as Decision Trees or CNNs—to categorize different brain tissue types. Once tissue labels are assigned, unsupervised clustering and voxel-based calculations are used to compute volumetric ratios of gray matter, white matter, and cerebrospinal fluid. Finally, the system generates visualizations, reports, and comparative metrics that provide a clear interpretation of segmentation accuracy and tissue volume distribution.

3.1 Layer Definition

Table 17: Layers Definition

Layers	Description
Presentation Layer	This layer provides the user-facing interface, including dashboards for hospital admins and doctors. It enables doctor registration, MRI image uploads, viewing analysis results, and downloading reports through a web-based UI.
Layers	Description

Business Logic Layer	This layer handles the system's core operations, including doctor registration, MRI processing, feature extraction, PCA, model prediction, and volume calculation. It also manages validation, access control, and coordination between the interface and the database.
Database Layer	This layer stores all persistent data, including hospital and doctor accounts, uploaded MRI images, preprocessing outputs, extracted features, PCA components, and generated reports. It also maintains entity relationships and supports fast data retrieval for dashboards and analysis history.

3.1.1 Presentation Layer

The Presentation Layer serves as the primary interaction point between the researcher and the system. It provides an intuitive user interface that allows users to register, log in, upload MRI slices, and monitor the preprocessing and segmentation workflow. This layer also displays visual outputs such as tissue maps, volumetric ratios, performance metrics, and downloadable reports. Its main purpose is to ensure a smooth and accessible experience while effectively presenting complex analytical results.

3.1.2 Application / Business Logic Layer

The Application or Business Logic Layer is responsible for executing the core computational workflow of the system. It handles all major processing tasks, including MRI preprocessing, feature extraction, dimensionality reduction through PCA, and the application of clustering algorithms. This layer also manages the training and execution of machine-learning and deep-learning models such as Decision Trees and CNNs to classify tissue regions. Finally, it computes volumetric estimates of gray matter, white matter, and CSF, ensuring accurate and

reliable analytical results.

3.1.3 Data Layer

The Data Layer is responsible for managing all data-related operations within the system, ensuring reliable storage and retrieval of information. It maintains the MRI datasets, associated metadata, preprocessing outputs, extracted feature matrices, and trained machine-learning models. This layer also stores volumetric results, evaluation metrics, and system logs, providing a structured and secure repository for all analytical components. Its primary role is to support efficient data access and ensure the integrity of the system's computational workflow.

3.2 Software Architecture

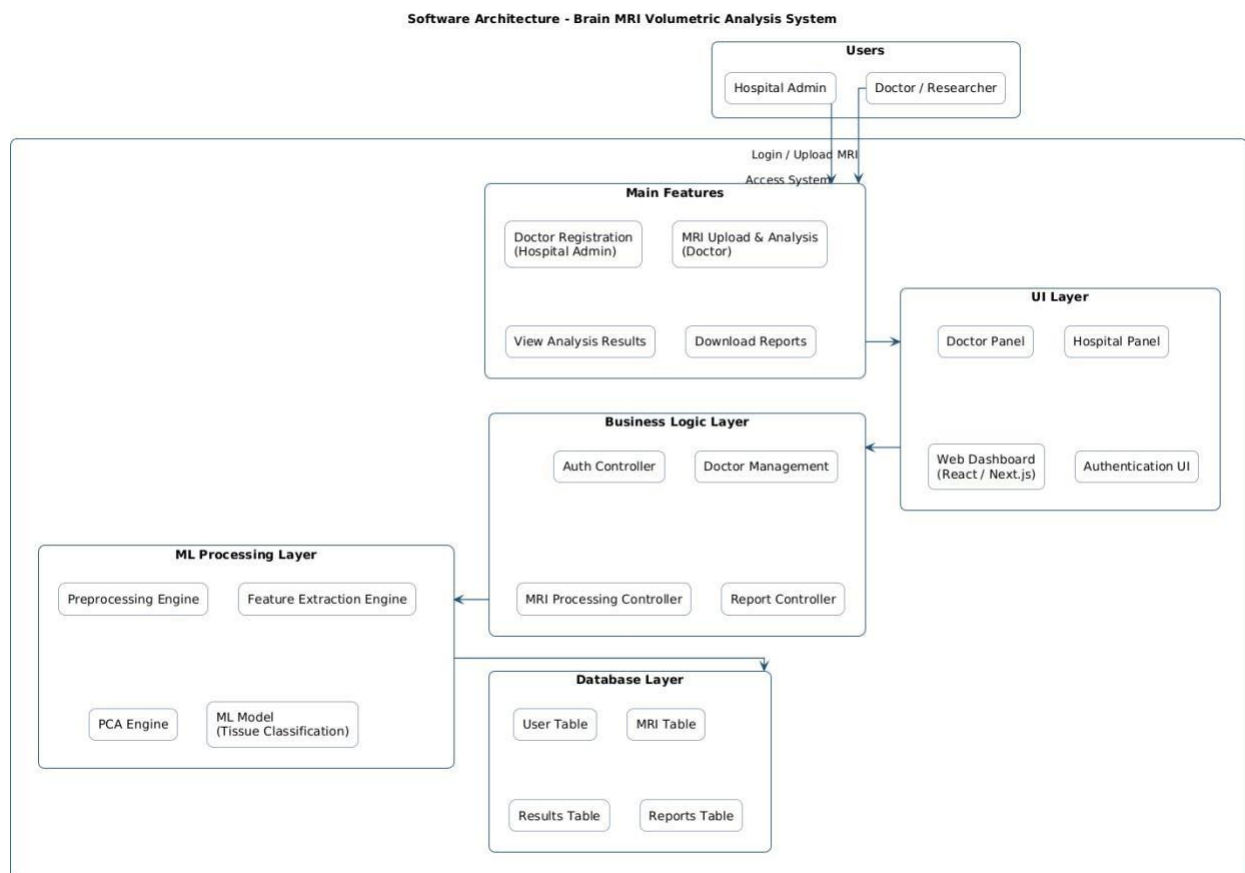


Figure 3.1 : Software Architecture Diagram

3.3 Class Diagram

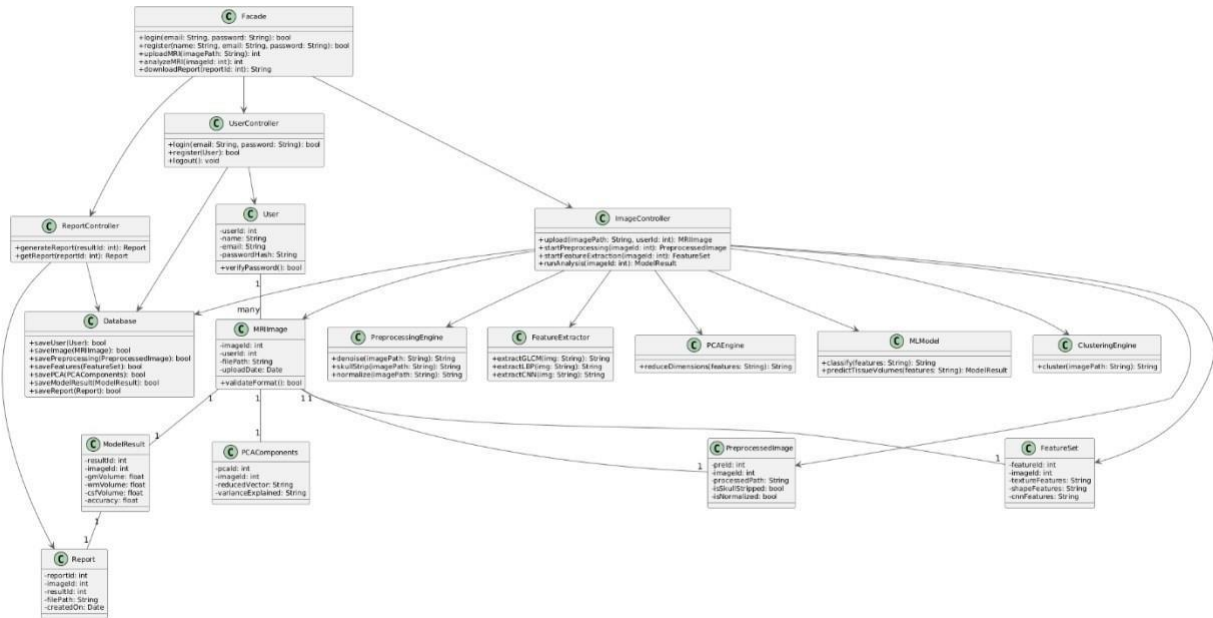


Figure 3.2: UML Class Diagram

- **Facade**

Provides a simplified interface for login, signup, and image upload operations. Coordinates interactions between controllers and core processing components.

- **UserController**

Handles user-related operations such as login and registration requests. Validates user credentials and communicates with the Database class.

- **User**

Represents a doctor or system user with basic account details. Provides functions to verify and update user passwords.

- **Image controller:**

Manages MRI upload requests and triggers preprocessing, feature extraction, PCA, and model prediction.

Acts as the main coordinator for running the complete analysis pipeline.

- **Report controller:**

Generates reports after analysis is completed and retrieves stored reports from the database.

Provides access to downloadable volumetric and segmentation summaries.

- **Database:**

Stores and retrieves users, MRI images, preprocessing results, feature sets, PCA outputs, and model results. Ensures persistent data handling for the entire system.

- **MRI image:**

Represents an uploaded MRI image with associated metadata. Stores preprocessing status and volumetric computation details.

- **Preprocessing engine:**

Handles denoising, skull stripping, intensity normalization, and contrast enhancement. Outputs a cleaned and standardized image for further processing.

- **Preprocessed image:**

Stores intermediate results after preprocessing, including denoised and normalized images. Acts as input for feature extraction and PCA.

- **Feature extractor:**

Extracts radiomic (GLCM/LBP), shape-based, and CNN deep features from MRI images. Generates a structured feature set for classification.

- **Feature set:**

Represents the collected features for a given MRI image. Stores texture, shape, and CNN-derived features in unified form.

- **PCA engine:**

Applies dimensionality reduction on extracted features. Generates PCA components that improve model efficiency.

- **PCA components:**

Stores principal components, explained variance, and reduced feature vectors. Used as input for classification models.

- **ML model:**

Executes classification using Decision Tree or CNN models. Predicts tissue categories and produces final model results.

- **Model result:**

Contains predicted tissue labels, accuracy, Dice score, and IoU results. Represents the evaluation outcome for a single MRI analysis.

- **Clustering engine:**

Performs unsupervised clustering (e.g., K-Means) to group tissue regions. Supports the volumetric estimation based on pixel clusters.

- **Report:**

Stores the final volumetric report, including percentages and performance metrics. Provides formatted results ready for export or visualization.

3.4 Sequence Diagram

Sequence Diagram model the flow of logic within your system in a visual manner enabling you both to document and validate your logic, and are commonly used for both analysis and design purposes.

3.4.1 Researcher Registration

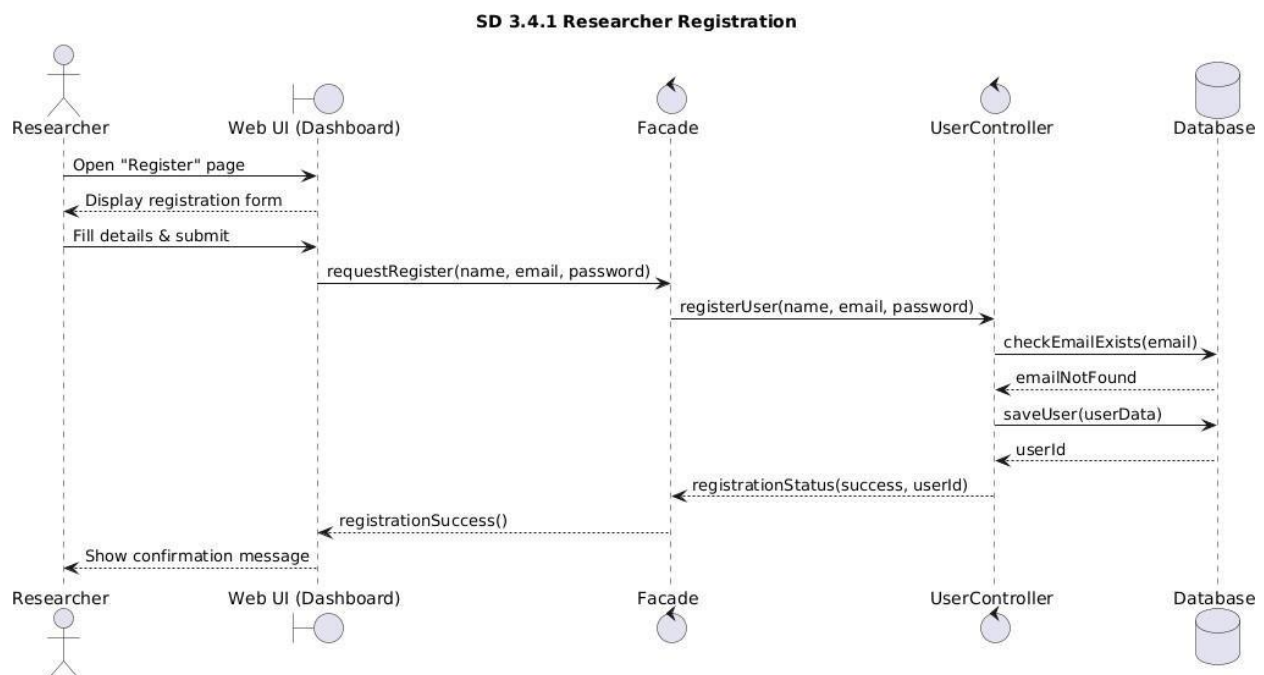


Figure 3.3: Researcher Registration

3.4.2 Researcher Login

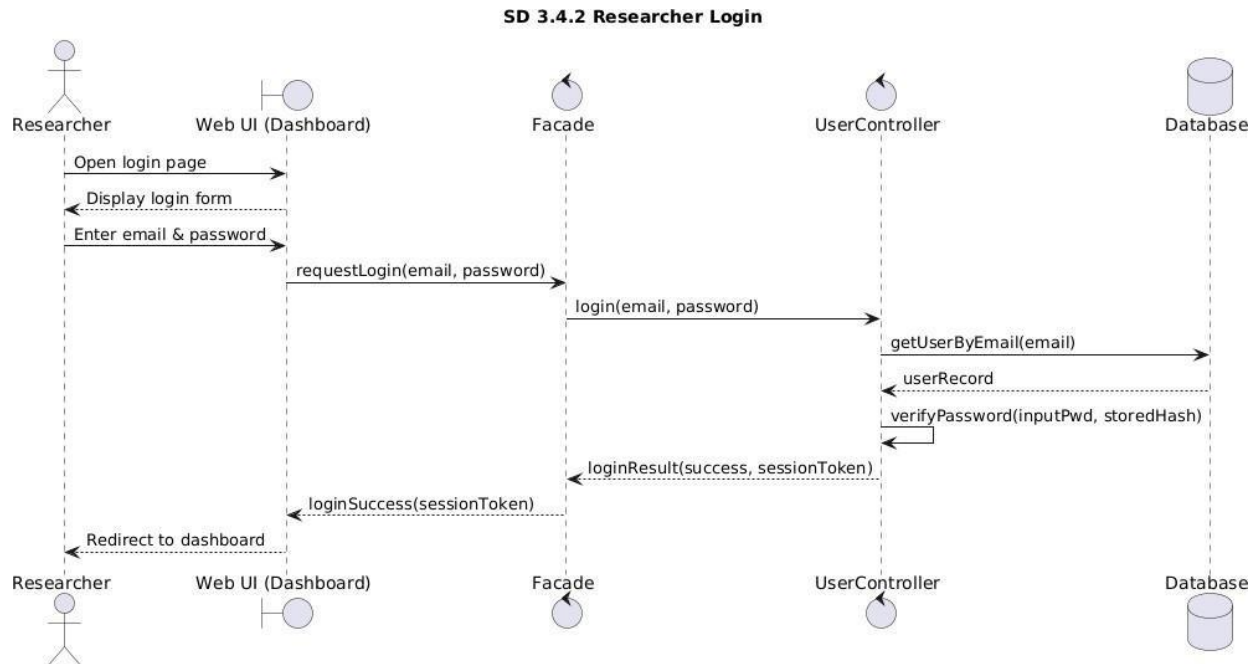


Figure 3.4: Researcher Login

3.4.3 (A) Upload MRI Image & Start Preprocessing

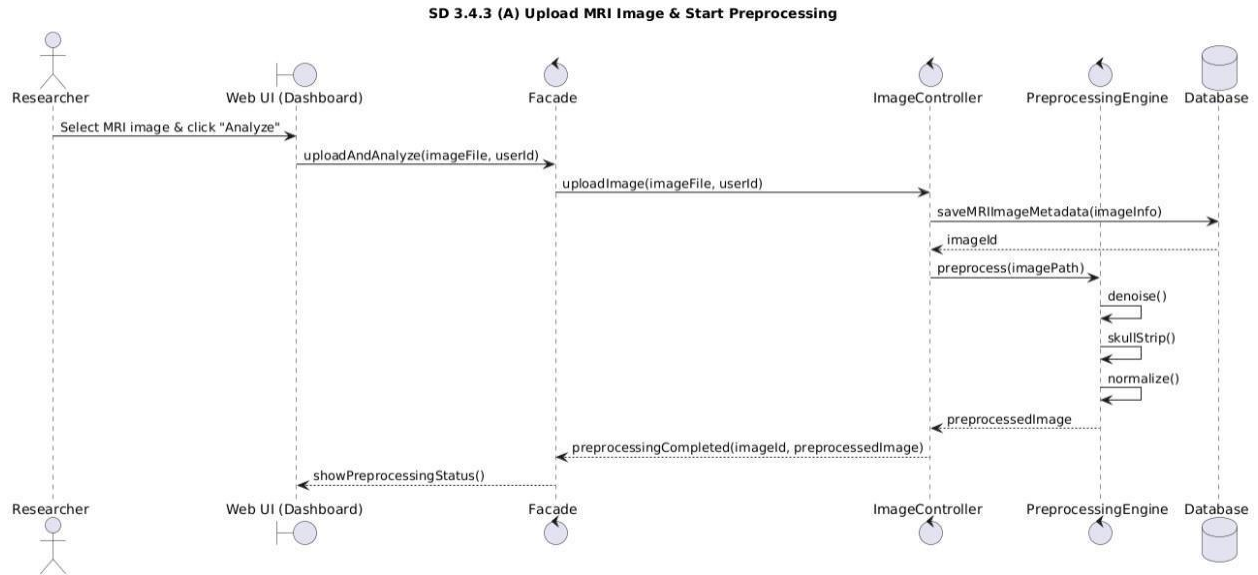


Figure 3.5: Upload MRI Image And Start Processing

3.4.3 (B) Feature Extraction, PCA, Classification & Volumetric Analysis

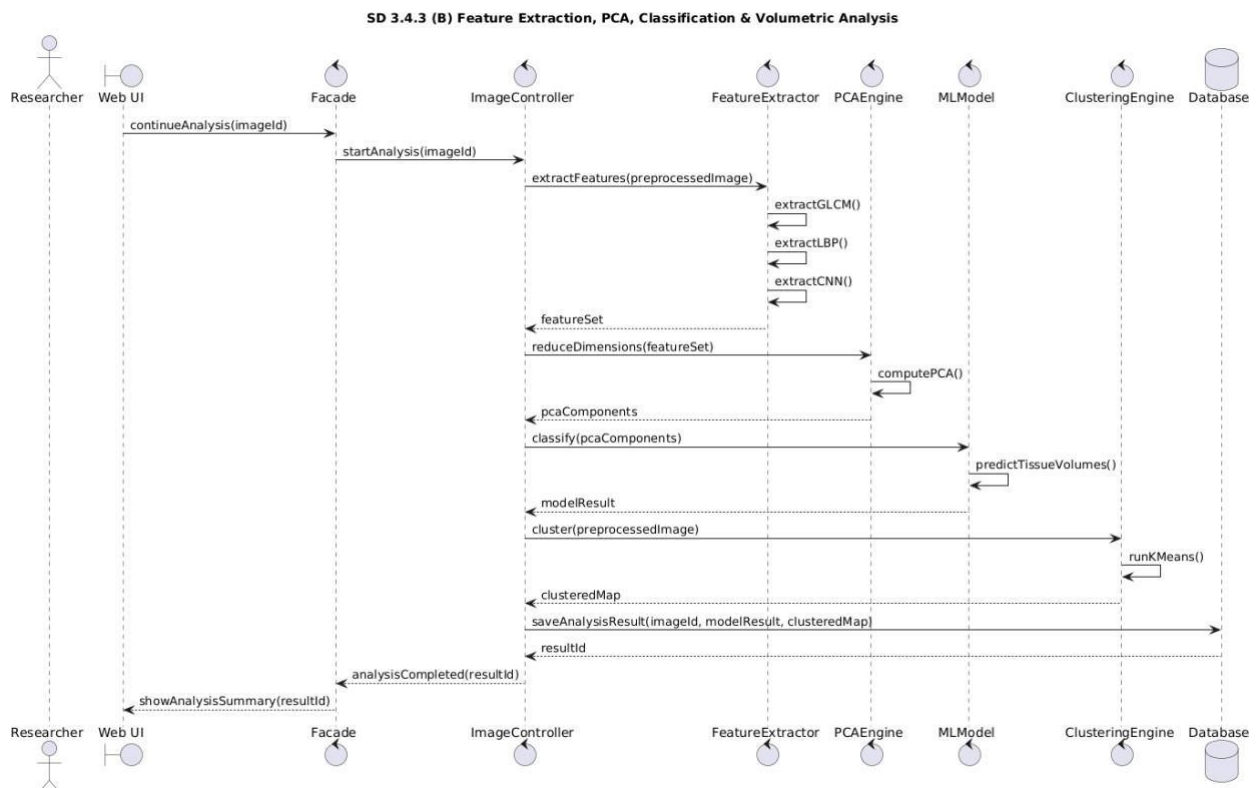


Figure 3.6: Feature Extraction , PCA, Classification & Volumetric Analysis

3.4.4 View & Download Volumetric Report

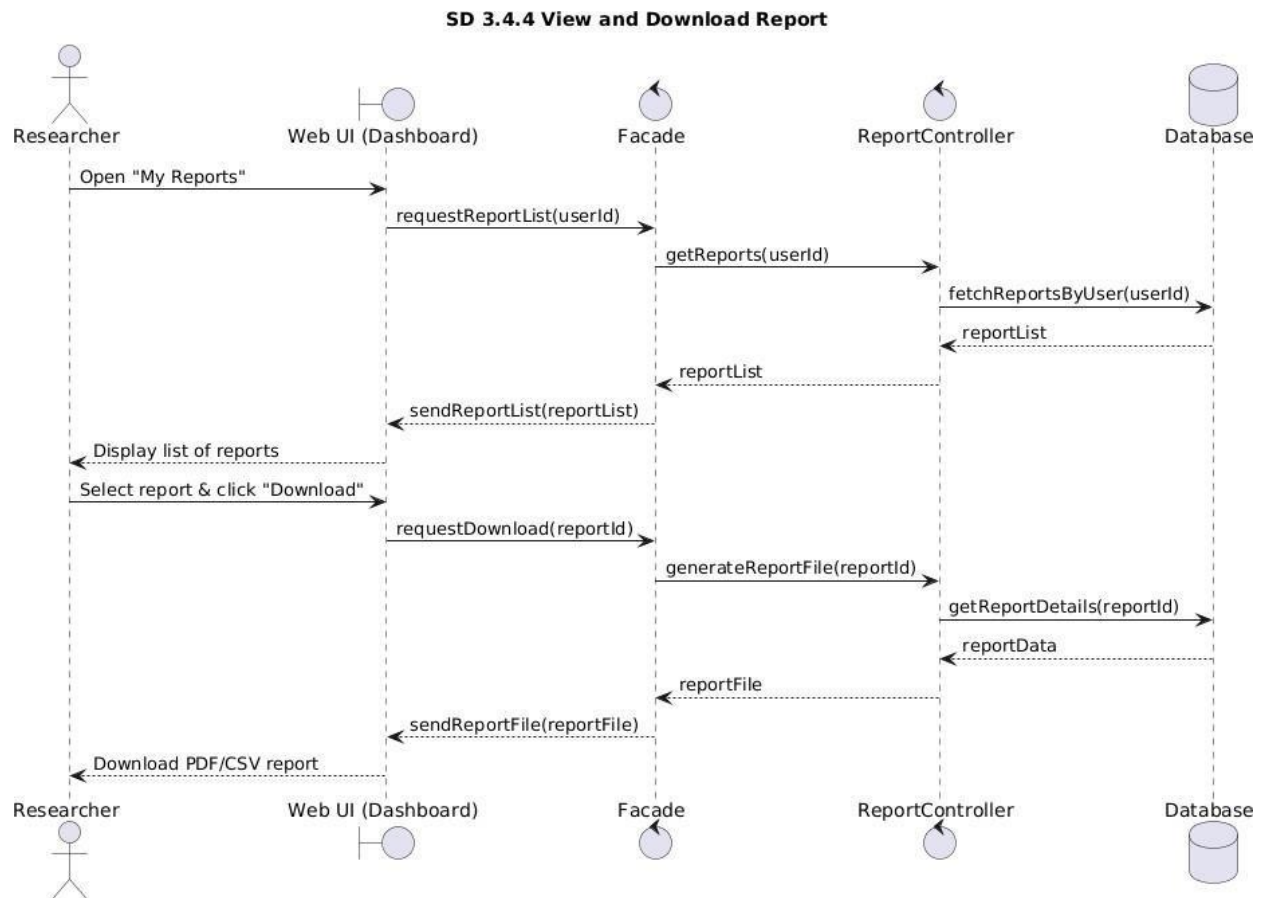


Figure 3.7: View & Download Report

3.5 Entity Relationship Diagram

ERD - Brain Soft Tissue Volumetric Analysis System

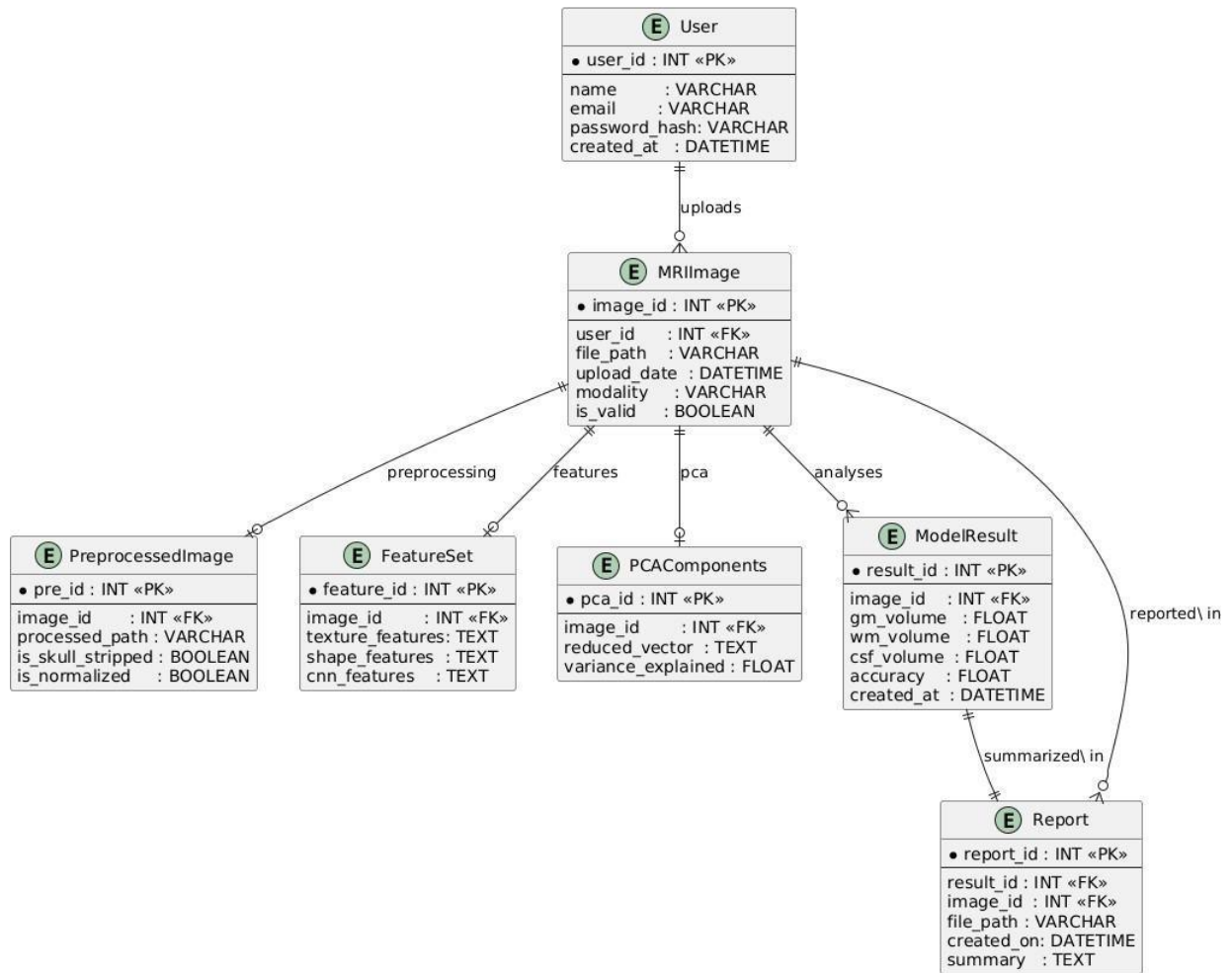


Figure 3.8: ERD Brain Soft Tissue Volumetric Analysis System

3.6 Database Schema

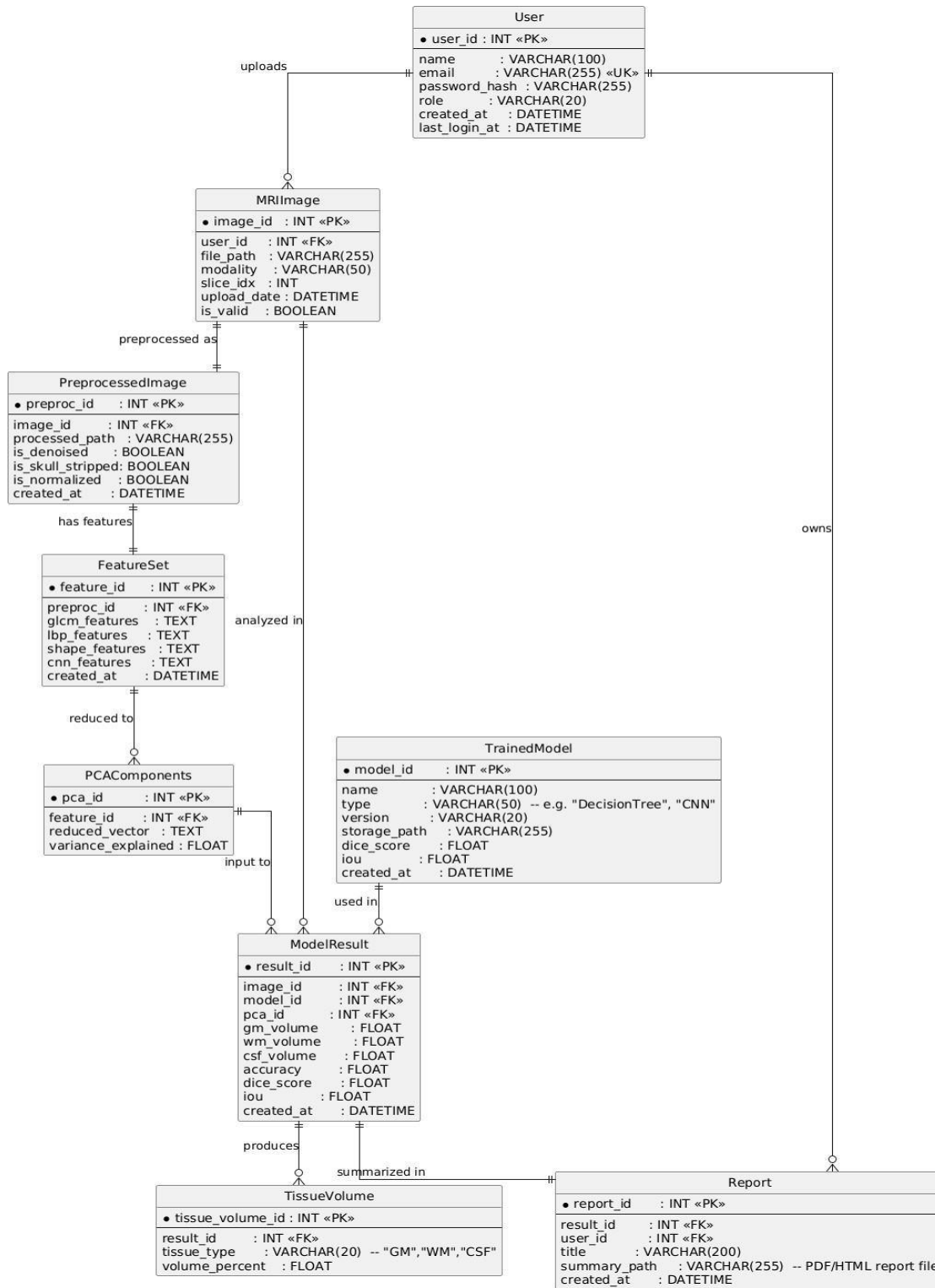
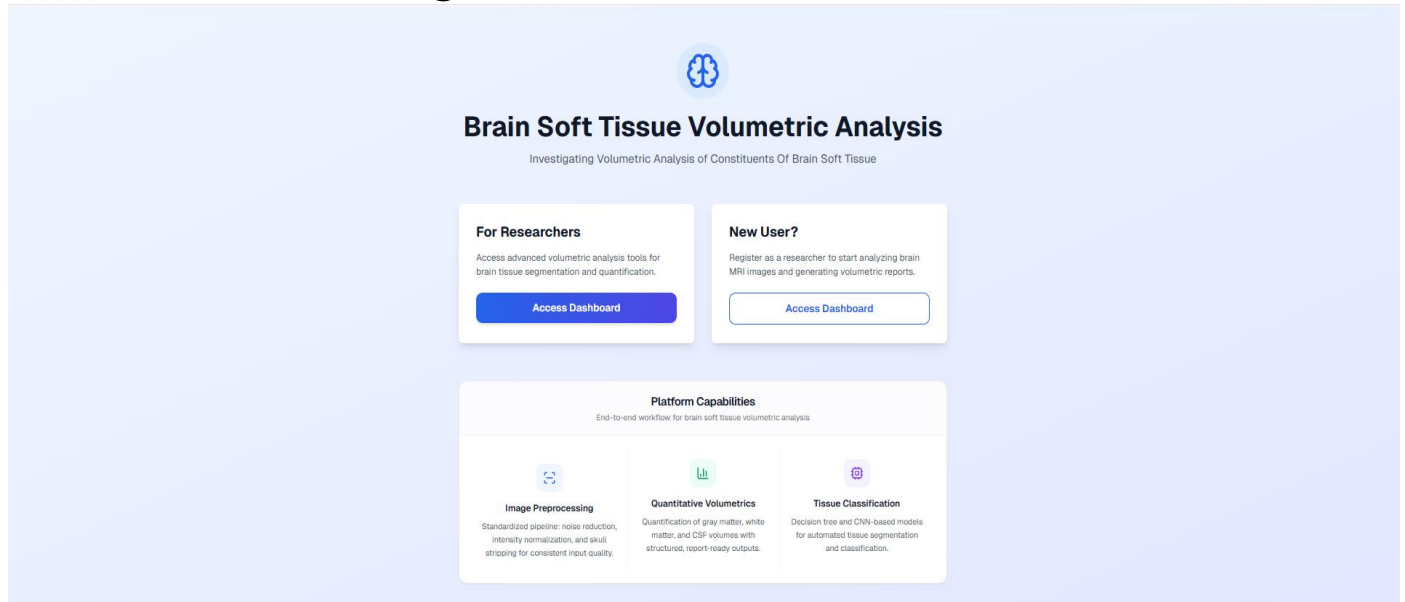
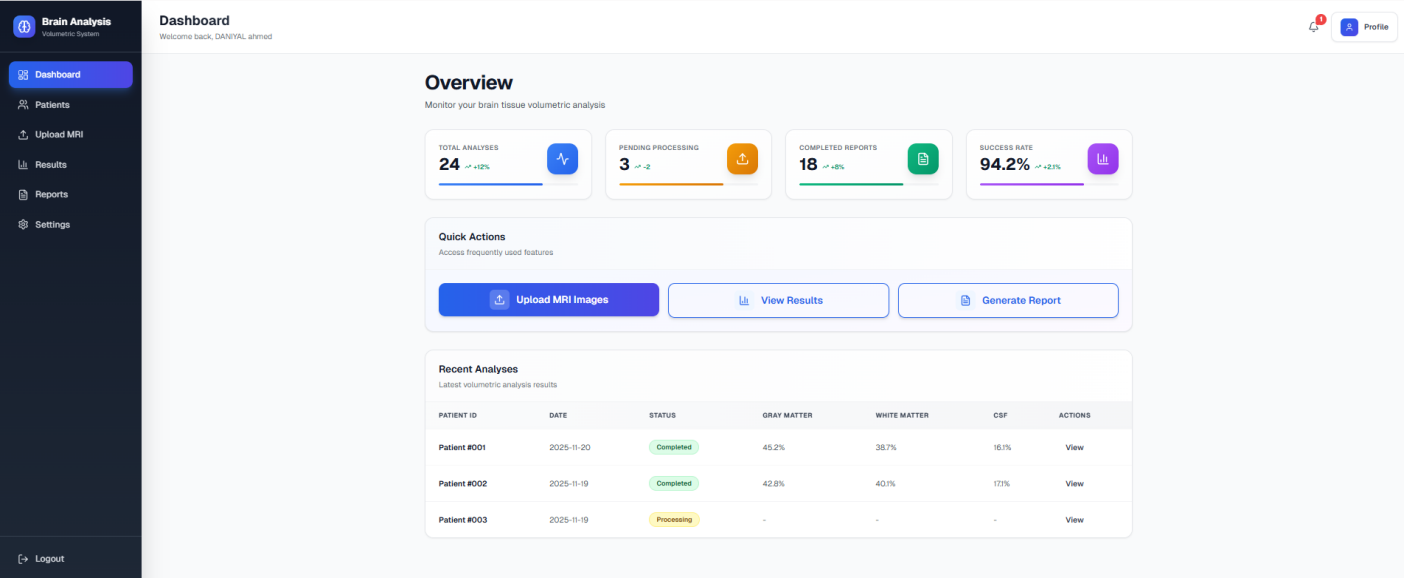


Figure 3.9: Database Schema for Brain Soft Tissue Volumetric Analysis

3.7 User Interface Design





4 Chapter 4:Implementation Issues

4.1 Coding Standards

The Brain Soft Tissue Volumetric Analysis System was developed using a consistent set of coding standards across the frontend and backend. These standards were followed to keep the codebase readable, maintainable, and aligned with common industry practice.

4.1.1 Indentation

The project used spaces for indentation rather than tabs. “The backend (Python) used 4 spaces per indentation level. The frontend (Next.js/TypeScript) used 2 spaces per indentation level, following common JavaScript and TypeScript practice.”. Nested blocks were indented by one extra level. This made control flow and block structure easy to see and reduced inconsistencies when working across different editors and platforms.

4.1.2 Declaration

Variables and functions were declared explicitly. In the backend, type hints were used where appropriate for function parameters and return values to clarify expectations and improve maintainability. In the frontend, TypeScript was used so that key data structures and API responses were typed. Constants were declared where values were reused and not meant to change, and mutable state was kept to a minimum to avoid unintended side effects.

4.1.3 Statement Standards

Statements were generally kept to a single line where possible. Long expressions were split across multiple lines with clear indentation. Conditional and loop bodies were always placed in dedicated blocks rather than on the same line as the condition. Early returns were used to handle error cases and invalid inputs before the main logic, which helped keep functions shorter and easier to follow. Comments were used to clarify non-obvious logic rather than to restate the code.

4.1.4 Naming Conventions

Descriptive names were used for variables, functions, and files. In the backend, functions and variables followed snake_case (e.g. perform_skull_stripping, denoise_method). Class names used PascalCase (e.g. PreprocessingPipeline). In the frontend, camelCase was used for variables and functions (e.g. processedImage, denoiseMethod), and PascalCase for React components and types. File names reflected their purpose: preprocessing logic lived under preprocessing, utilities under utils, and API routes under app/api. This made it easier to locate and understand each part of the system.

4.2 Development Enviroment

The development of the Brain Soft Tissue Volumetric Analysis System was carried out using a modern and flexible development environment. The system was developed on a Windows-based operating system using Visual Studio Code (VS Code) as the primary integrated development environment.

Visual Studio Code was selected due to its lightweight architecture, fast performance, and extensive support for modern web and backend technologies. It provides built-in support for JavaScript, Python, and JSON files, which were extensively used in the project. Its extension ecosystem allowed seamless integration of debugging tools, code formatting utilities, and version control support.

The frontend of the system was developed using React and Next.js within VS Code, enabling efficient component-based user interface development. For backend processing, Python scripts were written to perform MRI image preprocessing tasks such as resizing, normalization, noise reduction, and skull stripping.

The development environment also supported terminal integration, allowing the execution of backend scripts and dependency management directly within the editor. This setup improved productivity and reduced development time by enabling real-time testing and debugging.

Overall, the selected development environment provided flexibility, maintainability, and scalability, making it suitable for implementing a research-oriented medical imaging system.

4.3 Database Management System

The Brain Soft Tissue Volumetric Analysis System used a structured data management approach to store and manage system-related information. MongoDB was used as the database, with Mongoose as the object-document mapping library. The database stored user account details, such as firstname, lastname, username, email, contact number, PMDC and specialization, as well as patient records, including patient identifier, demographic information, medical history, and notes. Each patient record was linked to the user who created it via an uploadedBy reference, so that data could be associated with the corresponding researcher or clinician account.

Because of the research-oriented nature of the system, MRI image files were not stored in the database. Images were sent to the backend for preprocessing and were either processed in memory or saved temporarily on the server. Processed images were written to a processed/ directory on disk. Preprocessing results, such as processed image data and processing steps, were returned to the frontend through the API and displayed in the user interface but were not persisted in the database. Only user and patient information were stored in MongoDB. This approach kept storage requirements low and avoided storing large image data in the database.

The system supported insertion and retrieval of user and patient data. User registration and login, as well as patient registration and listing, were implemented through API routes that performed these operations. Deletion of records was not implemented in the current version.

Security was considered in the database design. User passwords were hashed using bcrypt before storage, rather than stored in plain text. Access to sensitive functionality, such as creating patients and uploading images, was limited to authenticated users. The frontend used the logged-in user's identity when creating patient records and when using the upload workflow. This helped protect the confidentiality of user and patient data and reduced the risk of unauthorized access.

Overall, the database management system provided structured storage for users and patients, kept user-patient associations clear, and supported the current workflow. The design allowed for future extension, such as storing classification or volumetric analysis results, if those features were added later.

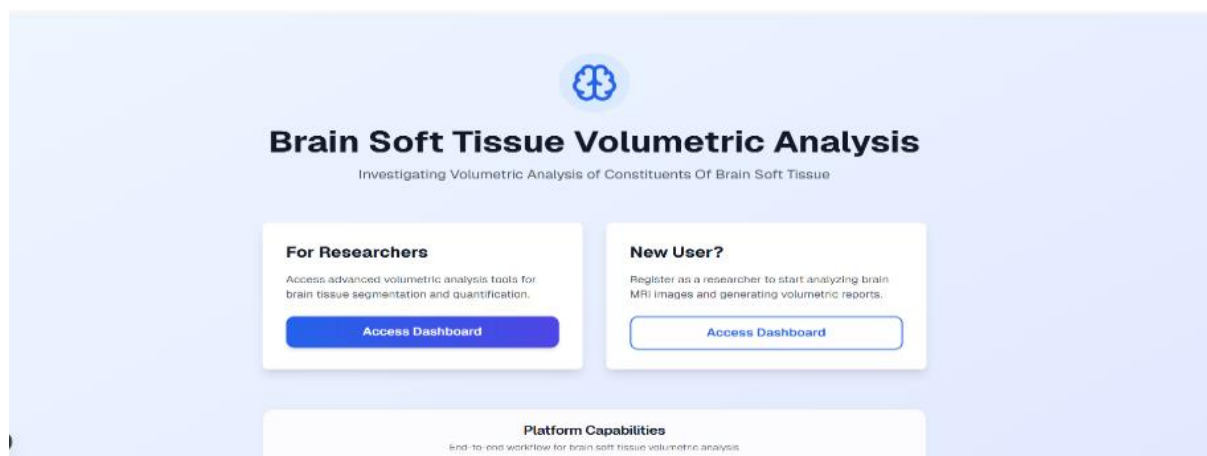
4.4 Software Description

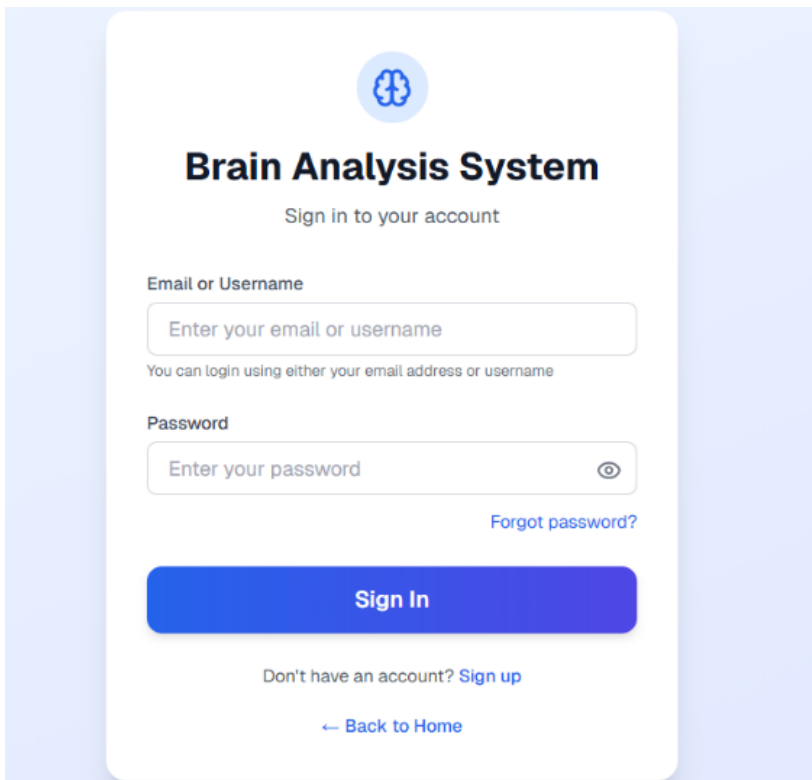
The Brain Soft Tissue Volumetric Analysis System was implemented as a web-based application consisting of a frontend and a backend. The frontend allowed users to register, log in, manage patients, and upload MRI images for preprocessing. The backend received uploaded images, applied a fixed sequence of preprocessing steps, and returned the results. The following subsections describe the software modules that handled image upload, preprocessing, and visualization of the preprocessed output.

4.4.1 User Authentication Module

Input:

- User registration details (firstname ,lastname, username, email, password, specialization)
- Login credentials (email/username and password)







Brain Analysis System

Sign in to your account

Email or Username

You can login using either your email address or username

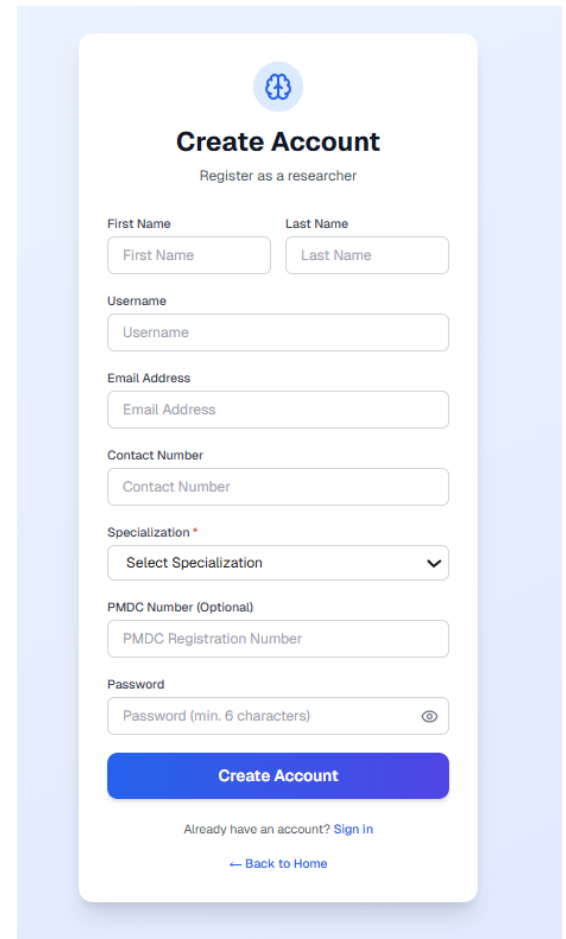
Password

[Forgot password?](#)

[Sign In](#)

Don't have an account? [Sign up](#)

[← Back to Home](#)





Create Account

Register as a researcher

First Name

Last Name

Username

Email Address

Contact Number

Specialization ^{*}

PMDC Number (Optional)

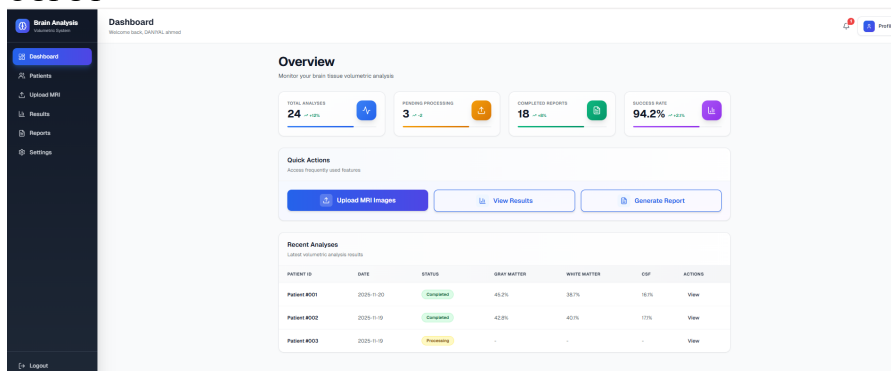
Password

[Create Account](#)

Already have an account? [Sign in](#)

[← Back to Home](#)

OUTPUT



Dashboard

Monitor your brain tissue volumetric analysis

Overview

Monitor your brain tissue volumetric analysis

Metric	Value	Change
TOTAL ANALYSES	24	+10%
PENDING PROCESSING	3	-2%
COMPLETED REPORTS	18	+5%
SUCCESS RATE	94.2%	+0.5%

Quick Actions

Access frequently used features

[Upload MRI Images](#) [View Results](#) [Generate Report](#)

Recent Analyses

Latest volumetric analysis results

PATIENT ID	DATE	STATUS	GRAY MATTER	WHITE MATTER	CSF	ACTIONS
Patient #001	2023-11-01	Completed	45.2%	38.7%	16.1%	View
Patient #002	2023-11-05	Completed	42.8%	40.1%	17.1%	View
Patient #003	2023-11-10	Pending	-	-	-	View

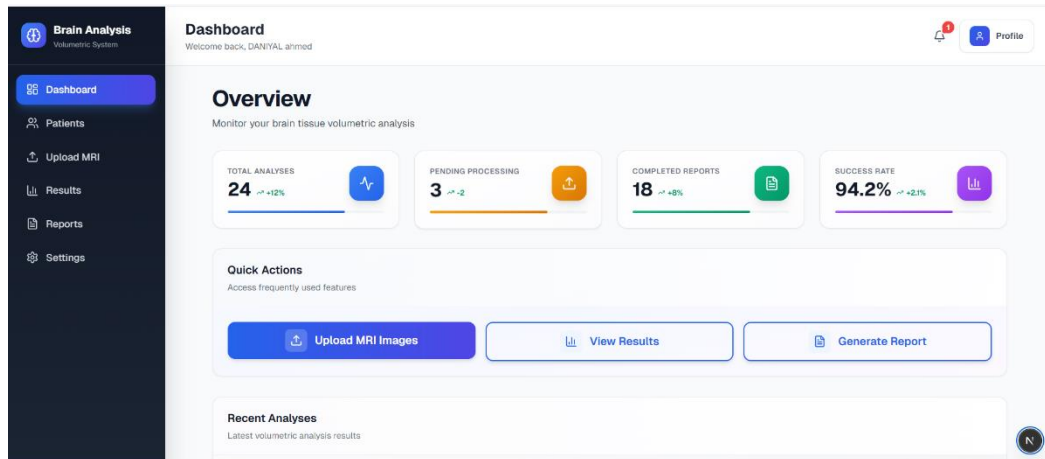
- Successful login and redirection to dashboard
- Error message on invalid credentials
- User session created on successful authentication

4.4.2 Dashboard Interface

Input:

- Authenticated user session
- User navigation actions (menu selection, quick links)

Output:



- Dashboard layout displayed with sidebar and header
- Access to patients, upload, results, reports, and settings modules

4.4.3 MRI Image Upload Module

Input:

- Selected or newly created patient
- MRI image files (JPG, PNG, DICOM, NIFTI)

The screenshot shows the 'Upload MRI Images' form. It has a sidebar with a menu containing 'Dashboard', 'Patients', 'Upload MRI', 'Results', 'Reports', and 'Settings'. The main content area has a header with 'Dashboard' and 'Welcome back, DANIAL ahmad'. Below the header is the 'Upload MRI Images' section. It has a sub-header 'Patient Information' and a form with fields for 'First Name', 'Last Name', 'Age', 'Gender', 'Contact Number', 'Email Address (Optional)', 'Address (Optional)', 'Medical History (Optional)', and 'Notes (Optional)'. There is a 'Next: Upload Images' button at the bottom right.

Output:

Patients
View and manage all registered patients

Patient Details ← Back to List

Patient ID PAT-MKOAHXJR-XJBZ	Contact Number 03205997146
Full Name DANIYAL GUL	Email dmn7146@gmail.com
Age 24 years	Address house 16 street 3 Jinnah garden phase 1 islamabad
Gender Male	Registered Date 1/21/2026, 10:21:18 PM

Patient History

No visits or uploads recorded yet. Upload MRI Images to create history.

[Upload MRI Images](#) Status: pending

Upload Images

Uploading images for Patient ID: DANIYAL GUL
Patient ID: PAT-MKOAHXJR-XJBZ
[View Patient Details](#)

Upload MRI Images Back

Upload images for Patient ID: PAT-MKOAHXJR-XJBZ

Visit / check-up date
01-Feb-2026

Notes for this visit (optional)
e.g. follow-up, symptoms, referral...

Drag and drop files here
or click to browse files
Supported formats: DICOM (dcm), NIFTI (nii), PNG, JPG, JPEG

[Select Files](#)

[Process More Images](#) [Start New Patient](#)

Preprocessing Results (1 image) ✓ All images processed successfully!

✓ **Te-gl_0015.jpg** 10:43:04 AM

Original Size: 512 × 512 × 3 pixels	Processed Size: 256 × 256 pixels
---	--

Processing Steps Completed:

- ✓ Resize to 256x256
- ✓ Convert to grayscale
- ✓ Denoising (gaussian)
- ✓ Skull stripping
- ✓ Histogram equalization
- ✓ Intensity normalization (0-1)

Processed Image Location:
processed/Te-gl_0015_processed_20260201_104304.png

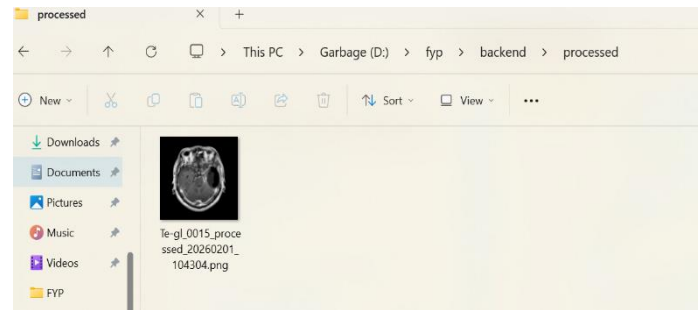
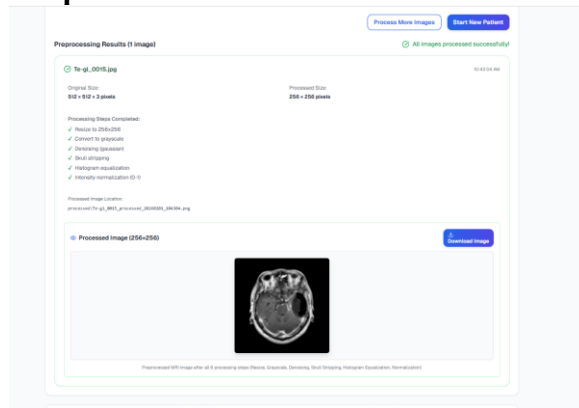
- Upload and processing status displayed
- Preprocessed image and related metadata shown
- Error message on invalid file or upload failure

4.4.4 Image Preprocessing Module

Input:

- Uploaded MRI image data

Output:



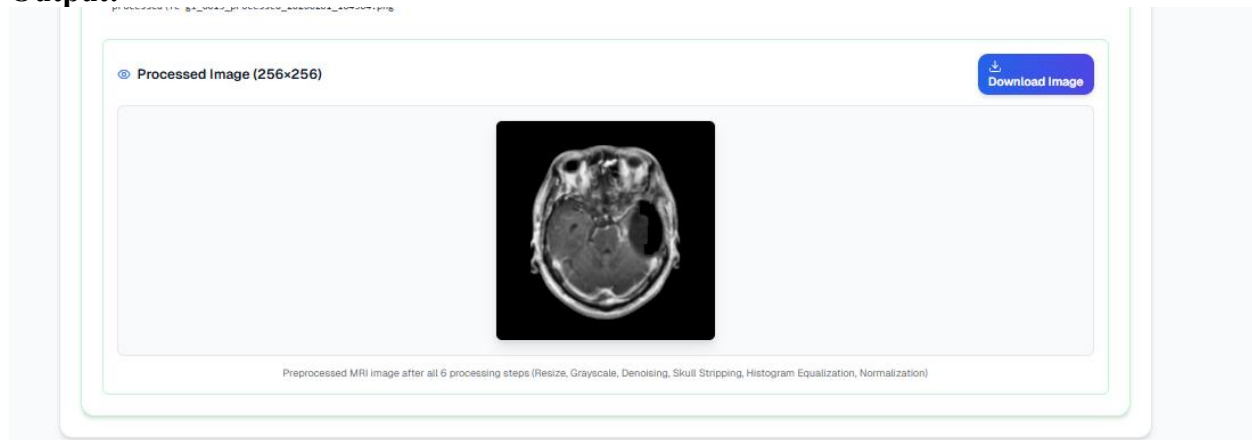
- Preprocessed MRI image (resized, denoised, normalized, skull-stripped)
- Processing metadata returned to frontend
- Image stored in server directory

4.4.5 Output Visualization Module

Input:

- Preprocessed image data
- Processing metadata (steps applied, dimensions)

Output:



- Display of preprocessed MRI image
- Visualization of applied preprocessing steps
- Option to view or download processed image

5 Chapter 5:Software Testing

5.1 Testing Methodology

Software testing was performed to ensure that the implemented system functions correctly and consistently produces reliable outputs under different operating conditions. As the project primarily focuses on frontend interaction and MRI image preprocessing, black-box testing methodology was selected because it allows effective validation of system functionality without requiring access to the internal code structure.

In black-box testing, the internal implementation of the system is not examined or analyzed. Instead, the system is tested by providing various valid and invalid inputs and then observing and evaluating the generated outputs. This testing approach is suitable for verifying functional behavior such as image upload, execution of preprocessing operations, and visualization of processed MRI images. It also helps confirm that the system meets the specified functional requirements from a user's perspective.

Each module of the system was tested independently to ensure that all components perform their intended tasks correctly. Test cases were carefully designed to validate system responses, check error handling capabilities, and verify the correctness and consistency of outputs. Special attention was given to handling incorrect inputs, unexpected user actions, and system failures to ensure stability.

The overall testing process ensured that the system behaves as expected under both normal and exceptional conditions. By validating functional accuracy, reliability, and robustness, the testing phase contributed to improving system quality and ensured a smooth and dependable user experience.

5.2 Testing Environment

The testing environment was configured to simulate real-world usage conditions and ensure reliable test results. All tests were conducted on a standard desktop system using a web browser to evaluate frontend functionality and MRI image preprocessing operations. The environment supported stable execution of system features and allowed consistent testing of system behavior under normal and error conditions.

5.3 Test Cases: User Sign-in

Table 5.1: User Sign-in

Date:	20 January 2026
System:	Brain Soft Tissue Volumetric Analysis System
Objective:	Verify user sign-in functionality
Test ID:	TC-01

Version:	1.0
Test Type:	Black Box Testing
Input:	Username = testuser Password = 123456
Expected Result:	User should be successfully authenticated and redirected to dashboard.
Actual Result:	User logged in successfully and dashboard displayed.
Status:	Pass

Description:

This test case was designed to verify the correct working of the user authentication module. The objective of this test was to ensure that registered users can access the system using valid login credentials. During testing, a valid username and password were provided through the login interface. The system successfully validated the credentials, authenticated the user, and redirected the user to the dashboard screen. This confirms that the sign-in functionality operates correctly and ensures secure access to the system.

5.3.1 Test Case: Invalid Sign In

Table 5.2: Invalid Login Attempt

Date:	20 January 2026
System:	Brain Soft Tissue Volumetric Analysis System
Objective:	Validate incorrect login handling
Test ID:	TC-02
Version:	1.0
Test Type:	Black Box Testing
Input:	Username = testuser Password = wrong123
Expected Result:	System should display an error message for invalid credentials.
Actual Result:	Error message displayed successfully.
Status:	Pass

Description:

This test case was performed to check the system's ability to handle invalid authentication attempts. Incorrect login credentials were entered intentionally to observe system behavior. The system correctly rejected the login request and displayed an appropriate error message. This test confirms that unauthorized access is prevented and proper validation mechanisms are implemented in the authentication module.

5.3.2 Test Case: MRI Image Upload

Table 5.3: Upload MRI Image

Date:	20 January 2026
System:	Brain Soft Tissue Volumetric Analysis System
Objective:	Upload MRI image for preprocessing
Test ID:	TC-03
Version:	1.0
Test Type:	Black Box Testing
Input:	MRI image file (.png / .jpg)
Expected Result:	MRI image should be uploaded successfully.
Actual Result:	Image uploaded without errors.
Status:	Pass

Description:

This test case was conducted to verify the MRI image upload functionality. A valid MRI image file was selected and submitted through the upload interface. The system accepted the image and confirmed successful upload. This test ensures that the system correctly handles valid medical image files and prepares them for preprocessing operations.

5.3.3 Test Case: Invalid File Upload

Table 5.4: Unsupported File Format

Date:	20 January 2026
System:	Brain Soft Tissue Volumetric Analysis System
Objective:	Validate unsupported file format
Test ID:	TC-04
Version:	1.0
Test Type:	Black Box Testing
Input:	File type = .pdf
Expected Result:	System should reject unsupported file format and display warning.
Actual Result:	File rejected and error message shown.
Status:	Pass

Description:

This test was executed to ensure that the system properly validates uploaded files. An unsupported file format was intentionally uploaded to test validation rules. The system correctly identified the invalid file type and prevented further processing. An appropriate warning message was displayed to guide the user. This test confirms the robustness of input validation and prevents incorrect data from entering the preprocessing pipeline.

5.3.4 Test Case: Image Preprocessing

Table 5.5: Preprocessing Execution

Date:	20 January 2026
System:	Brain Soft Tissue Volumetric Analysis System
Objective:	Execute MRI image preprocessing
Test ID:	TC-05
Version:	1.0
Test Type:	Black Box Testing
Input:	Uploaded MRI image
Expected Result:	Systems should preprocess the MRI image successfully.
Actual Result:	Preprocessed image generated successfully.
Status:	Pass

Description:

This test case was designed to verify the execution of the image preprocessing module of the image. After a valid MRI image was uploaded, the preprocessing pipeline was initiated. The system successfully performed resizing, grayscale conversion, noise reduction, intensity normalization, histogram equalization, and skull stripping. The output image showed improved contrast and reduced background noise, confirming correct execution of preprocessing operations.

5.3.5 Test Case: Output Visualization

Table 5.6: Display Preprocessed Image

Date:	20 January 2026
System:	Brain Soft Tissue Volumetric Analysis System
Objective:	Display preprocessing output
Test ID:	TC-06
Version:	1.0
Test Type:	Black Box Testing
Input:	Preprocessed image data
Expected Result:	Preprocessed image should be displayed correctly on the interface.
Actual Result:	Output displayed successfully.
Status:	Pass

Description:

This test case was conducted to verify the output visualization functionality of the system. After preprocessing, the resulting MRI image was displayed on the user interface. The system rendered the processed image without distortion or delay. This confirms that the frontend correctly receives processed data from the backend and presents it in a clear and user-friendly manner.